AI Memory

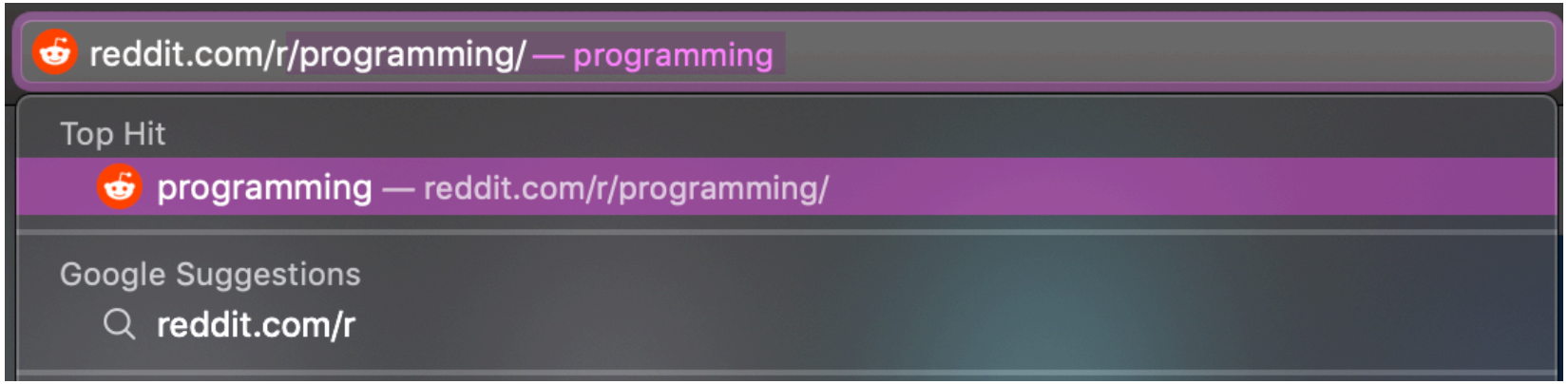
DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

What happens when you type a URL and press Enter?

`https://www.reddit.com/r/programming/`



A favorite interview question for new grads: one answer touches every layer of the system stack, from the browser down to the network and back up to the server.

A small action crosses a whole system

```
Address bar → DNS → connection → encryption → HTTP → server → page
```

- Each layer transforms the request into something the next layer understands
- Several layers can reuse previous work
- A slow page may be waiting on the network, the server, or the browser

Recall L1: identify the bottleneck before optimizing.

Six questions for today

1. What happens when you type a **URL**?
2. What happens when you type a **prompt into an agent**?
3. What is **memory** in an AI agent?
4. Why is that memory **finite**?
5. How do agents organize a **memory hierarchy**?
6. What does the **Claude Code source leak** reveal about its harness?

Q1 · Step 0: interpret the address

```
https://www.reddit.com/r/programming/
  scheme       host           path
```

- A URL names a destination; search terms go to a search engine
- The browser may suggest completions and speculatively resolve hosts
- Autocomplete, privacy settings, and browser version affect this

Some work may happen before you press Enter.

Step 1: turn a hostname into an address

```
; <<>> DiG 9.10.6 <<>> www.reddit.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54898
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;www.reddit.com.                        IN      A

;; ANSWER SECTION:
www.reddit.com.         61      IN      CNAME   reddit.map.fastly.net.
reddit.map.fastly.net.  30      IN      A       151.101.189.140

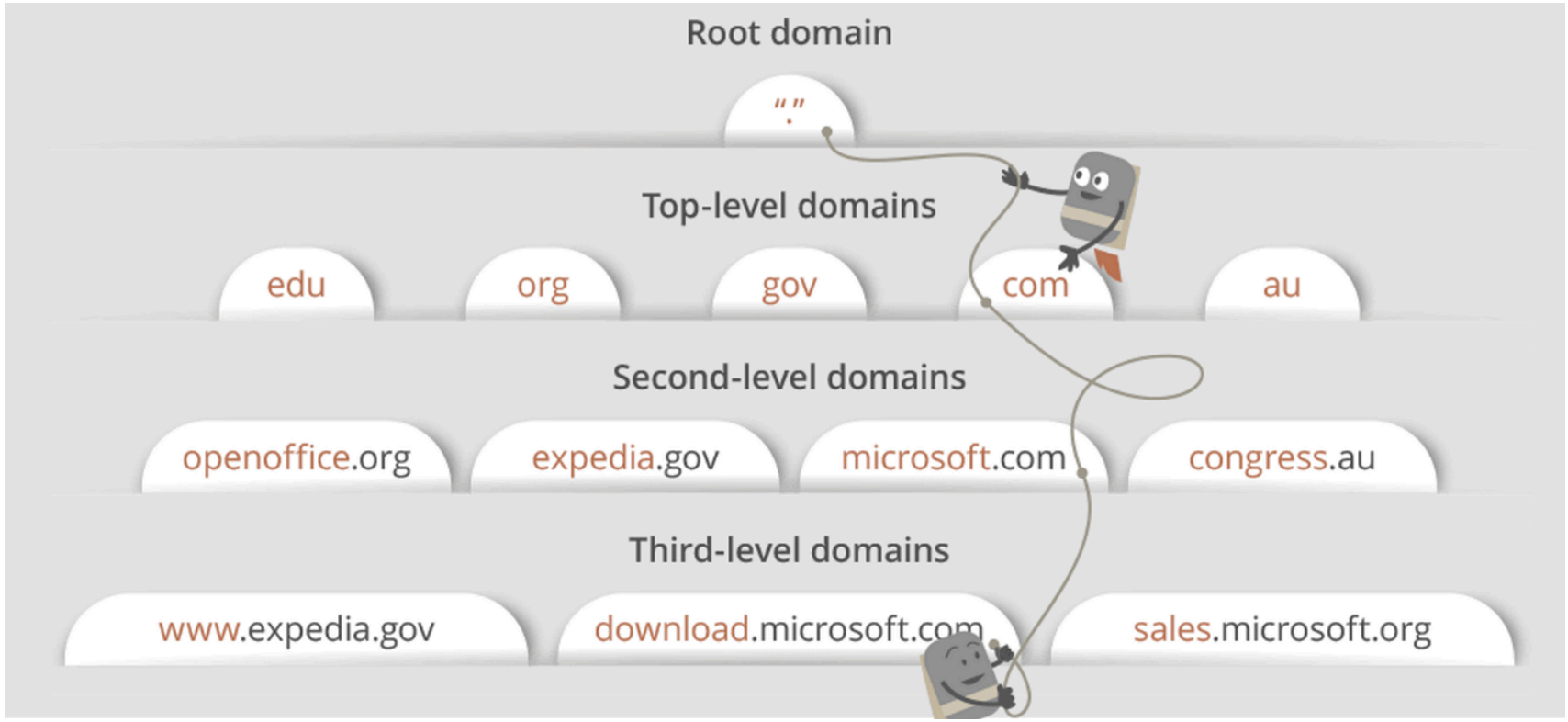
;; Query time: 162 msec
;; SERVER: 9.9.9.9#53(9.9.9.9)
;; WHEN: Mon Jul 08 14:20:03 AEST 2019
;; MSG SIZE  rcvd: 94
```

```
dig www.reddit.com A
```

- `reddit.com` is an alias (CNAME) for a CDN name, which has the address
- DNS records carry a **time to live (TTL)**
- A CDN may answer differently by location and time
- Run it live; do not memorize a particular Reddit IP

Figure: Louis Cremen, "What actually happens when you type a URL in the browser", Medium, 2019.

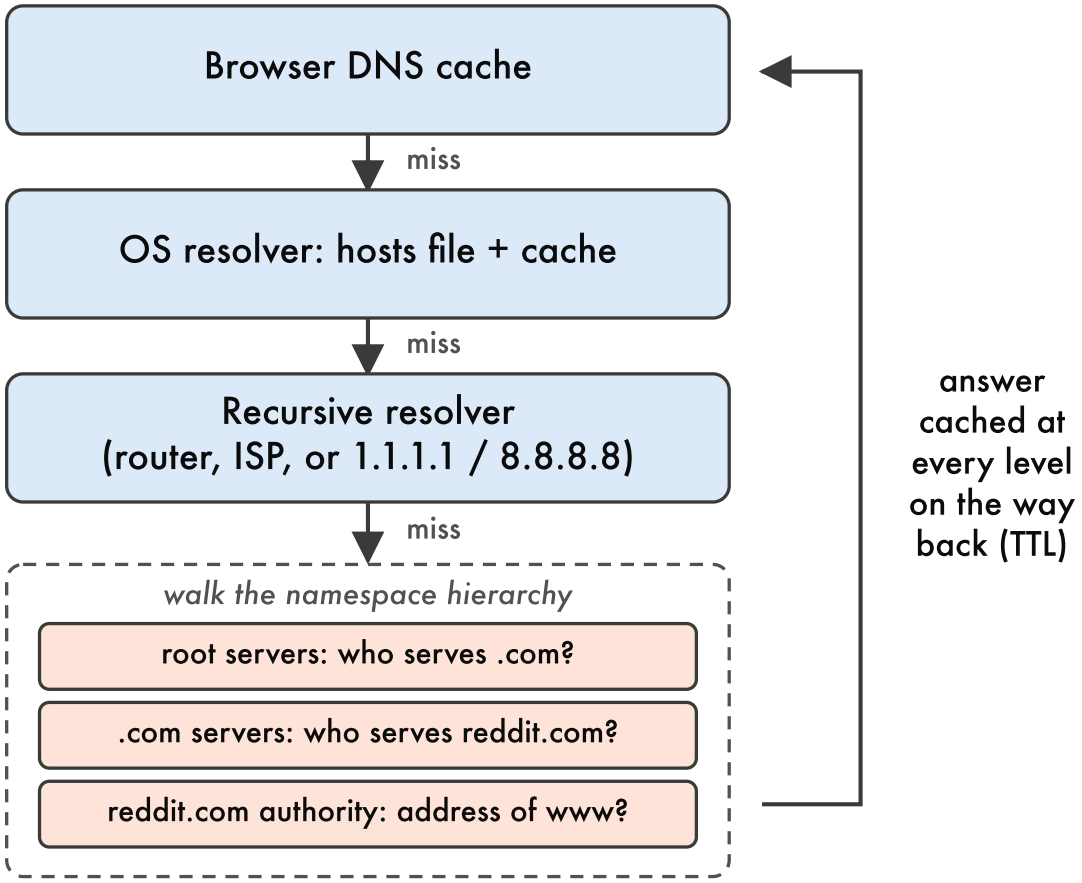
A hostname is a path through a namespace



Root → top-level domain → second-level domain → host. Each level is served by different servers.

Figure: Louis Cremen, "What actually happens when you type a URL in the browser", Medium, 2019.

DNS: where can the answer already be?

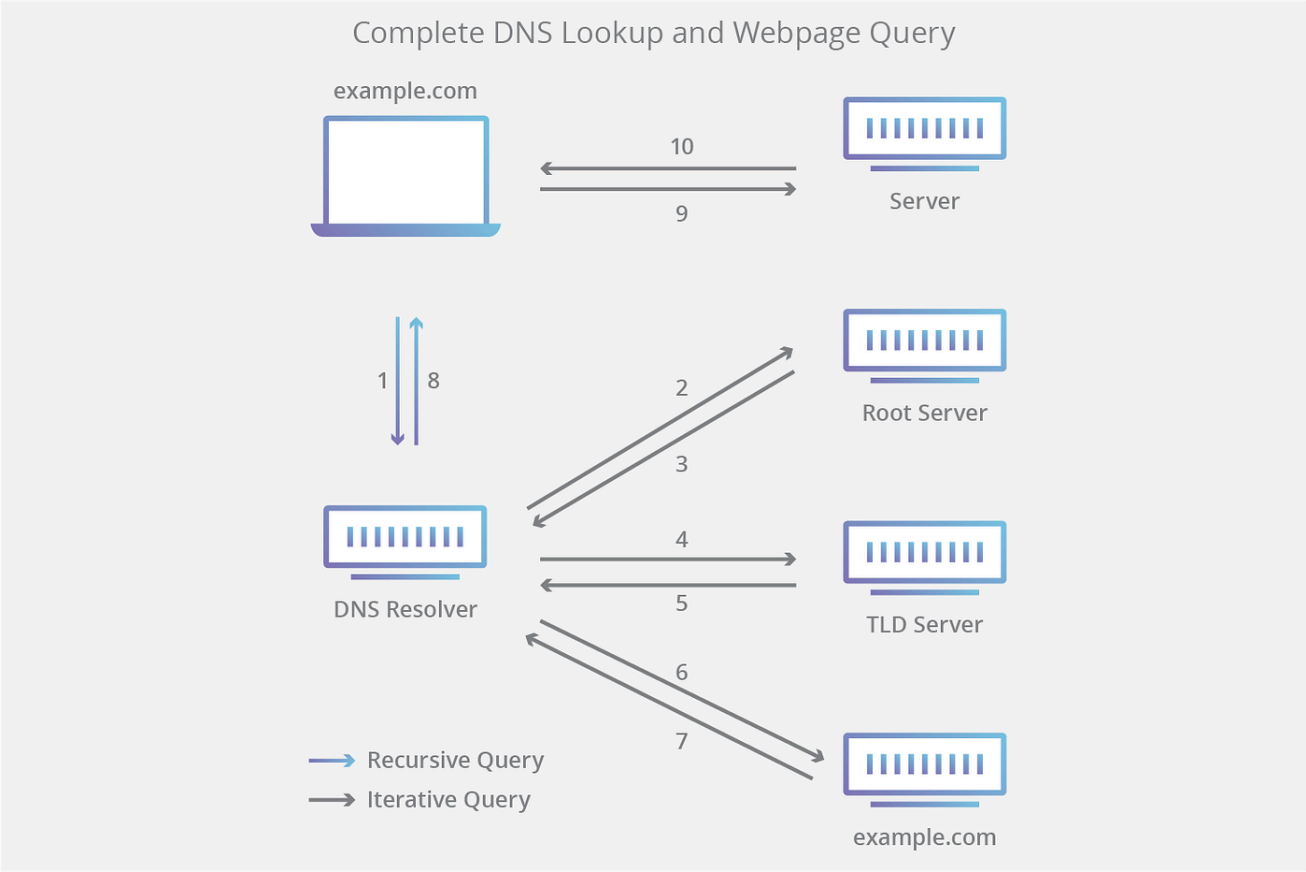


- Each level is asked only if the one above **misses**
- The answer is cached at every level on the way back, for its TTL
- Browser DNS-over-HTTPS can bypass the OS path
- A home router is not necessarily a DNS cache

A conceptual lookup path, not one universal implementation.

Cloudflare, DNS overview: https://www.cloudflare.com/learning/dns/what-is-dns/

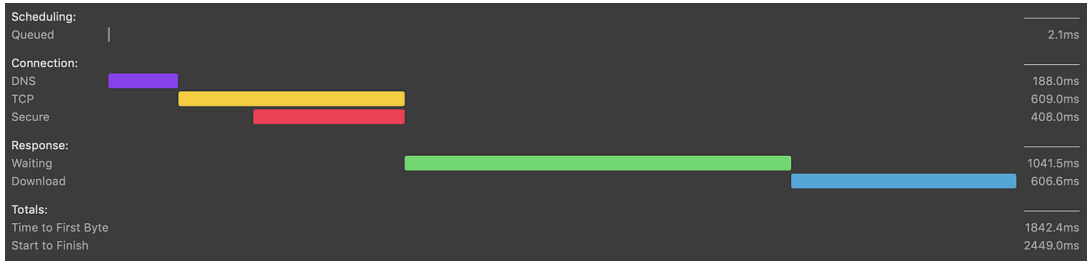
A cache miss: follow the DNS delegation



- Root: *who serves* `.com`*?*
- `.com` : *who serves* `reddit.com`*?*
- Authority: *what is the address of* `www.reddit.com`*?*
- There are **13 named root-server identities**, served by many instances
- The resolver caches answers and delegations; a warm resolver skips most of this

Diagram: Cloudflare Learning, "What is DNS?", via Cremen (2019). https://www.cloudflare.com/learning/dns/what-is-dns/

Demo: compare a cold and a warm lookup



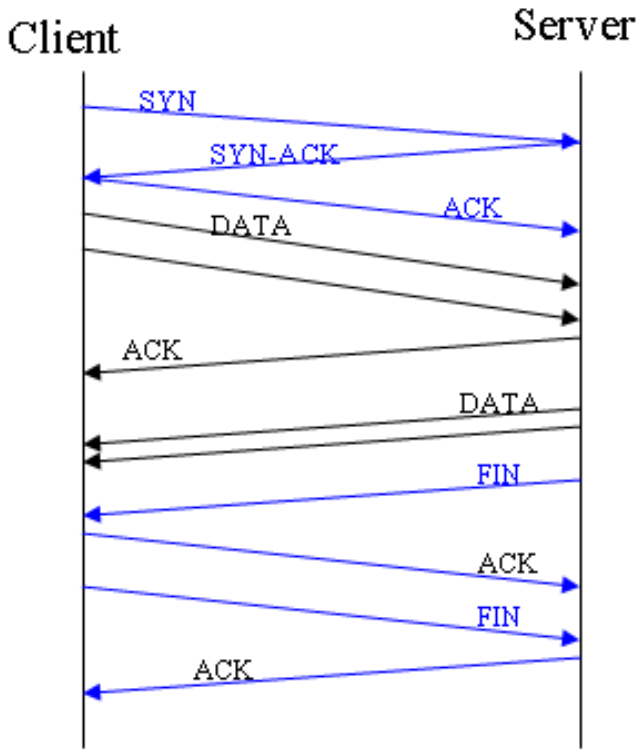


Observation	Possible explanation
No new DNS lookup	Cached answer or reused connection
No connection setup	Existing connection reused
Little or no transferred content	Browser cache or revalidation

An absent timing phase is evidence to investigate, not proof of one cache.

Figure: Louis Cremen, "What actually happens when you type a URL in the browser", Medium, 2019.

Step 2: establish a connection



For a new **HTTPS connection over TCP**, usually port 443:

- **SYN → SYN-ACK → ACK**, then data can flow
- TCP supplies an ordered, reliable byte stream
- An existing connection can avoid a new handshake
- **HTTP/3 uses QUIC over UDP** instead of this TCP path

Figure via Cremen (2019). TCP: https://www.rfc-editor.org/rfc/rfc9293.html · HTTP/3: https://www.rfc-editor.org/rfc/rfc9114.html

Step 2.5: authenticate and encrypt with TLS

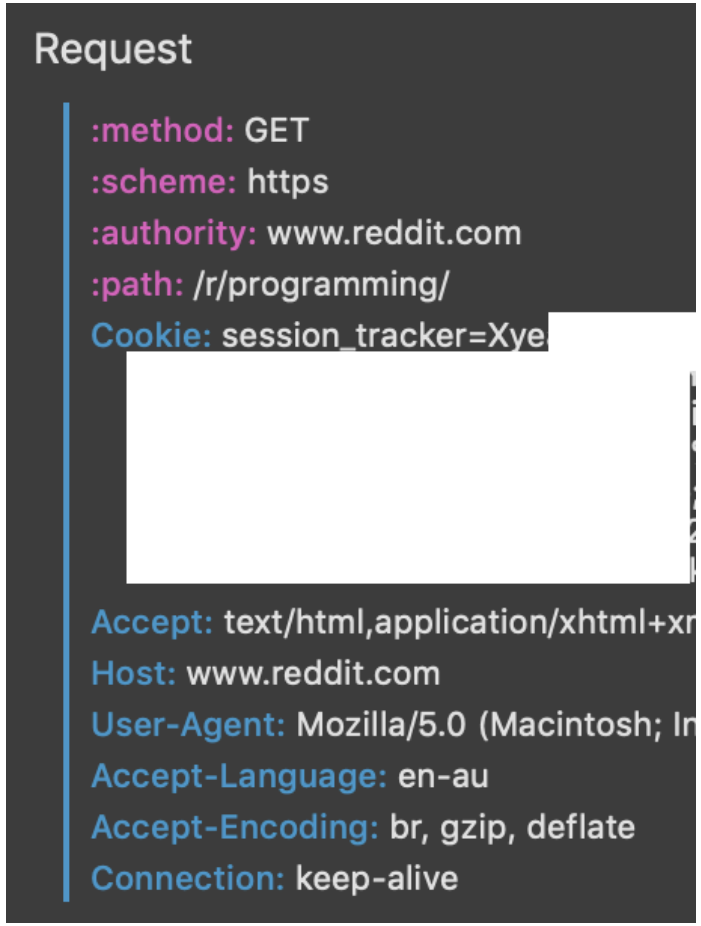
Simplified **TLS 1.3** handshake after TCP setup:

```
ClientHello + key share ──────────────────────────→
       ←──── ServerHello + key share
       ←──── encrypted certificate, proof, Finished
Client Finished ──────────────────────────────────→
             encrypted application traffic
```

- Validate the server certificate and negotiate traffic keys
- Session resumption can reduce setup work
- Do not mix this diagram with the older TLS 1.2 message sequence

TLS 1.3, RFC 8446: https://www.rfc-editor.org/rfc/rfc8446.html

Step 3: send an HTTP request



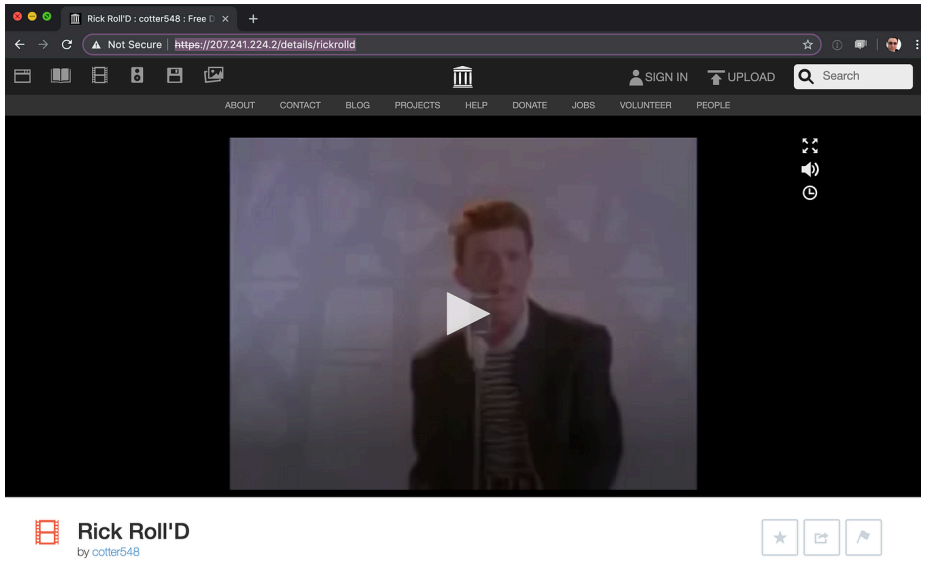

```
GET /r/programming/ HTTP/1.1
Host: www.reddit.com
Accept: text/html
```

- The host names the site; the path names the resource
- Several sites can share one IP address
- HTTPS also uses the hostname for certificate checks and TLS SNI
- HTTP/2 and HTTP/3 encode the same request differently

Screenshot via Cremen (2019). HTTP semantics: https://www.rfc-editor.org/rfc/rfc9110.html

Why the bare IP address fails: the Host header





`https://151.101.189.140/r/programming`
→ the CDN sees `Host: 151.101.189.140`
and cannot pick a site

`https://207.241.224.2/details/rickrolld`
→ one site per address, so it works

The IP finds the machine; the Host header finds the site.

Figure: Louis Cremen, "What actually happens when you type a URL in the browser", Medium, 2019.

Step 4: the server routes the request

```
Listen 80

# This is the "main" server running on 172.20.30.40
ServerName server.example.com
DocumentRoot "/www/mainserver"

<VirtualHost 172.20.30.50>
    DocumentRoot "/www/example1"
    ServerName www.example.com

    # Other directives here ...
</VirtualHost>

<VirtualHost 172.20.30.50>
    DocumentRoot "/www/example2"
    ServerName www.example.org

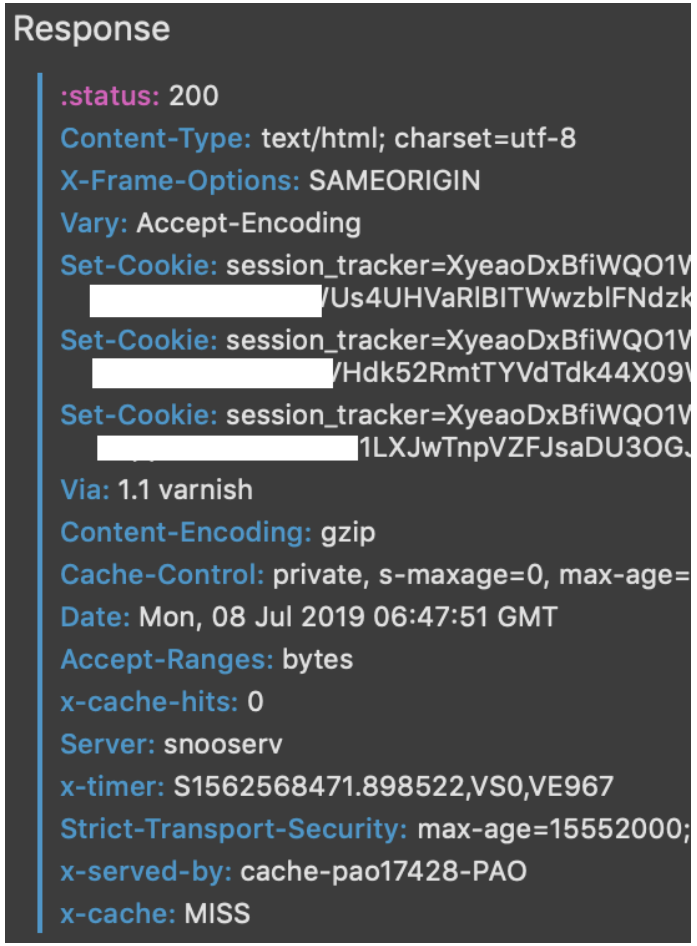
    # Other directives here ...
</VirtualHost>
```

```
CDN / reverse proxy → web server
  → application router → handler
```

- The web server matches the **Host** to a virtual host (Apache) or server block (nginx)
- The framework maps the **path** to a handler (Django `urls.py`)
- **A cache hit at the CDN can avoid reaching the application altogether.**

Apache VirtualHost example via Cremen (2019). https://httpd.apache.org/docs/2.4/vhosts/examples.html

Step 5: the server returns a response




```
HTTP/1.1 200 OK
Content-Type: text/html
Cache-Control: max-age=60

<html>...</html>
```

- Status line, headers, then the body
- `http://` would likely answer **301/302** to `https://`
- Headers tell the browser how long it may cache the body

Figure: Louis Cremen, "What actually happens when you type a URL in the browser", Medium, 2019.

Step 6: one page becomes many requests

```
HTML → parse document → discover CSS, scripts, images, fonts
                                  ↓
                          additional requests
                                  ↓
                        layout → paint → display
```

- Cached resources may avoid downloads
- Lazy-loaded images may be fetched later
- JavaScript can initiate more requests after the first render

“The HTML arrived” does not mean “the page is ready.”

Q2: What happens when you prompt a coding agent?

Claude Code · Codex · Cursor

“Find out why the tests fail, fix the bug, and verify the fix.”

Step 0: your input reaches a program

```
Your message → agent harness → model API
                        <->
               files, tools, state
```

- The **harness** is the application surrounding the model
- Your message is only part of the model input
- The application decides what context and capabilities are available

Step 1: assemble the model's workbench

```
Rules + tool definitions + relevant project information
      + conversation + retrieved observations + your request
                                ↓
                          model request
```

- The harness selects and formats these components
- Some content is always loaded; other content arrives on demand
- Exact ordering and message roles depend on the implementation

Context engineering is a systems task.

Anthropic, Effective context engineering for AI agents (2025): https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Instructions and memory arrive from several places

```
Organization instructions
User instructions: ~/.claude/CLAUDE.md
Project instructions: CLAUDE.md
Local preferences: CLAUDE.local.md
Scoped rules + recalled notes + conversation
                  ↓
          selected model context
```

Unlike DNS caches, these sources can add information rather than replace it.

Claude Code memory documentation: https://code.claude.com/docs/en/memory

Step 2: send a model request

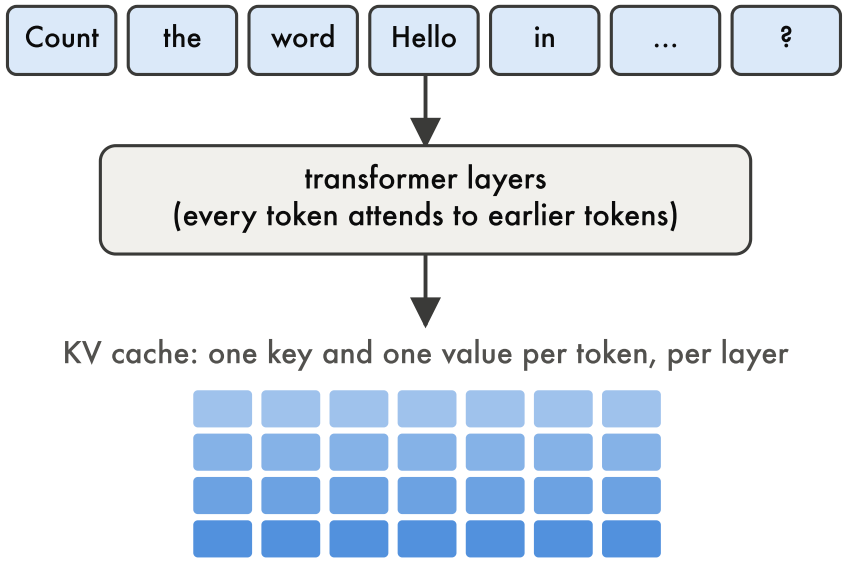
```
POST /v1/messages
Host: api.anthropic.com
Content-Type: application/json
```

```
{"model": "<model-id>", "max_tokens": 1024,
 "messages": [{"role": "user", "content": "Fix the tests"}],
 "tools": [{"name": "read_file", "input_schema": {"type": "object"}}]}
```

- Illustrative request; authentication and full schemas omitted
- HTTPS uses the same networking stack from Q1; connections may be reused

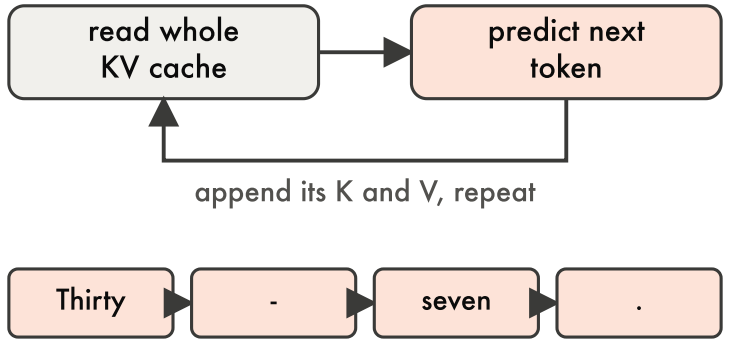
Step 3: prefill builds the KV cache

Prefill: all input tokens, in parallel



cost grows with input length: compute $O(n^2)$, memory $O(n)$

Decode: one output token per step

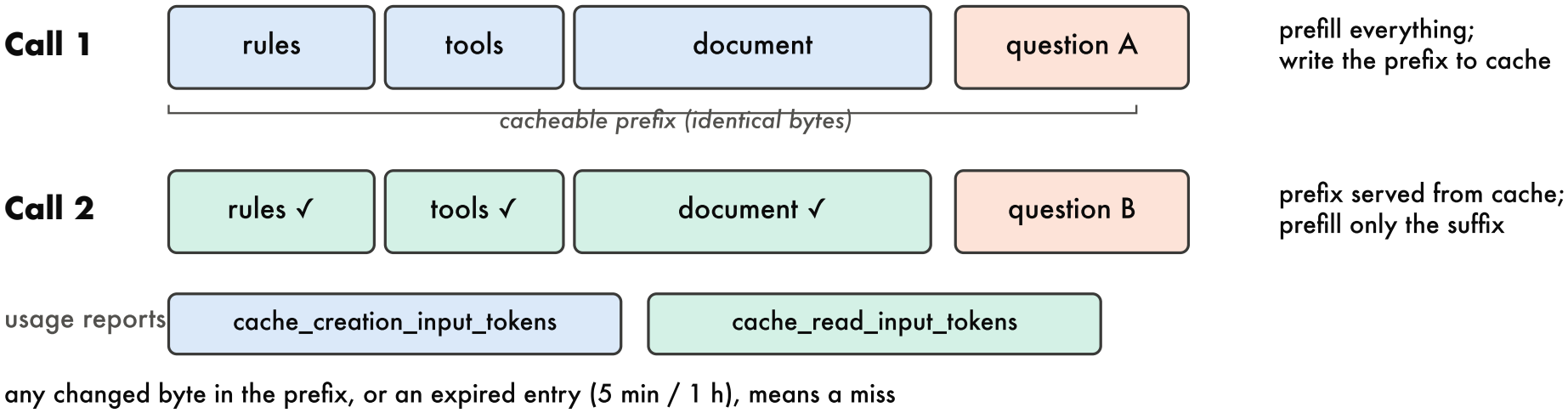


each step re-reads the cache:
long context → memory-bandwidth bound

streaming shows tokens as they are produced

- Prefill processes the whole prompt with substantial parallelism; decode is sequential
- Longer inputs increase compute (pairs) and storage (keys and values)

A matching prefix can skip repeated prefill



- Reuse requires matching eligible content under the provider's cache rules
- Inspect the usage counters rather than assuming a hit

Claude API prompt caching: https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Demo: count “Hello” in 12,000 words

```
python assets/llm_count_benchmark.py --dry-run
python assets/llm_count_benchmark.py \
  --models claude-haiku-4-5 --runs 5
```

Record per request	Why?
Time to first token (TTFT)	User-visible start latency
Total time	Complete response latency
Cache writes / reads	Did reuse actually occur?
Answer and token cost	Accuracy and resource use

API measurements pending; the second command uses credentials and incurs charges.

Demo: instruction first or text first?

```
A: [Count “Hello”] [12,000-word text]
B: [12,000-word text] [Count “Hello”]
```

Method	Correct count	Time
Local Python split/count	37	~0.28 ms
Local Python regex	37	~0.41 ms
Model × prompt order	Measure	Measure

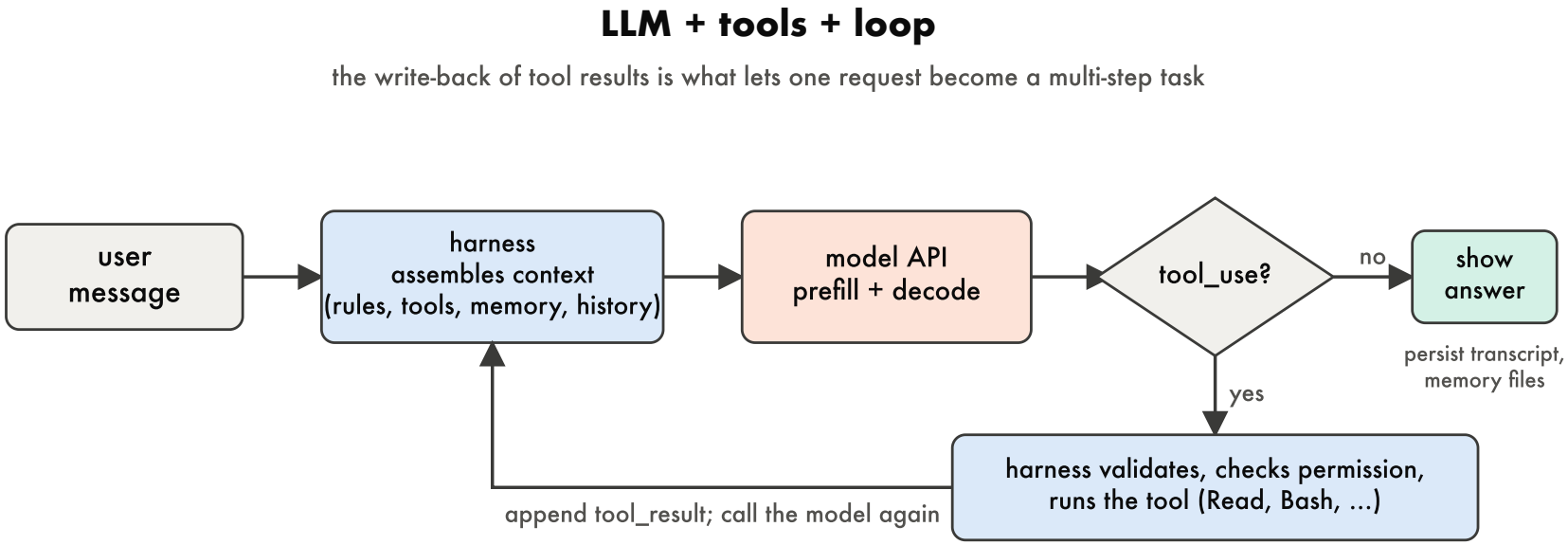
- Local results: best of 9, this machine, Oct. 14, 2026
- Compare both orders using the same text and repeated trials
- Exact counting is a good reason for an agent to call a deterministic tool

Step 4: decode generates the answer

```
Prompt KV cache
     ↓ predict next token
“Here” → “is” → “the” → “fix” → ...
     <-- update KV cache each step -->
```

- Standard autoregressive decoding generates successive tokens
- Each step uses prior inference state instead of rebuilding it from scratch
- Streaming displays partial output while generation continues
- Long contexts can make reading KV state a bandwidth bottleneck

Step 5: a tool request becomes an observation



- Tool arguments are requests, not executed actions; successes and errors both become observations

Executing a tool without feeding back its result breaks the loop.

The agent loop, stripped to its essentials

```
messages = [user_message]
for turn in range(MAX_TURNS):
    reply = call_model(messages, tools=tool_schemas)
    messages.append(reply)
    if not reply.tool_calls:
        break
    for call in reply.tool_calls:
        result = validate_authorize_execute(call)
        messages.append(tool_result(call.id, result))
```

- Pseudocode: the helpers hide substantial engineering
- Production systems also need budgets, cancellation, retries, and persistence

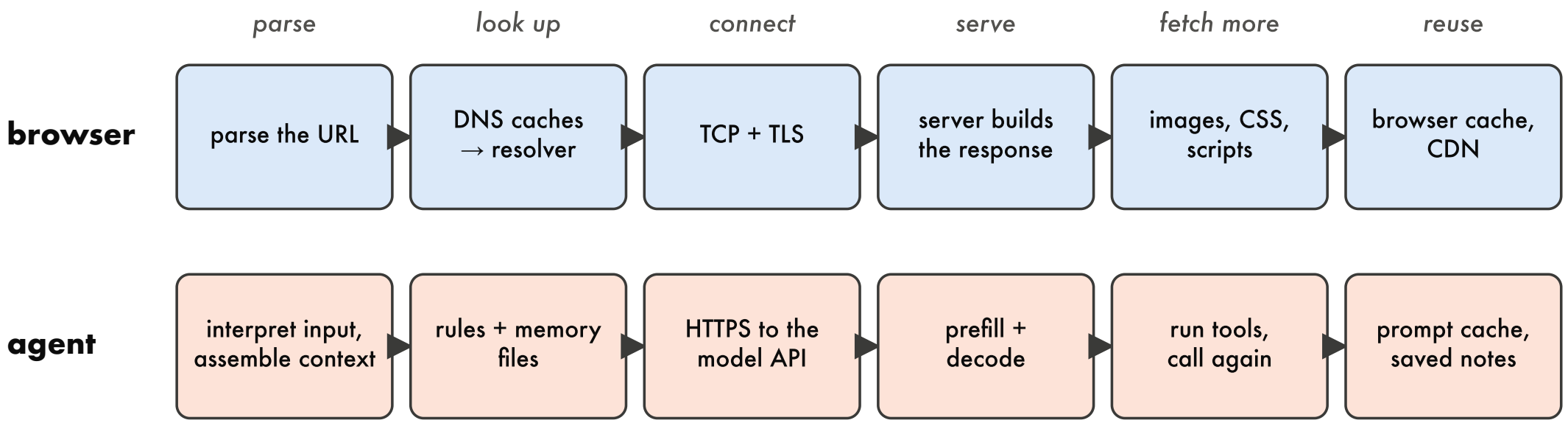
Step 6: preserve useful state

State	Possible destination
Full conversation / tool trace	Session log
Durable project facts	Memory files
Current task and progress	Application state
Condensed history	Compaction summary

- Recording a transcript does not mean every line is sent next turn
- Memory writes and compaction depend on the product and configuration

Persisting information and selecting information are different jobs.

URL request vs. agent request



same shape: a request descends through layers, and several layers can reuse earlier work

Similar systems pattern; different meanings of "cache."

Q3 · What does a model remember across sessions?

```
Session 1: “My name is Alice.” → “Hi, Alice!”
                    ↓ new request, no history or memory
Session 2: “What is my name?” → insufficient information
```

- Ordinary inference does not update model weights with your conversation
- A service can retain logs, notes, or cached state outside the model
- To use a saved fact, the system must supply it again or retrieve it

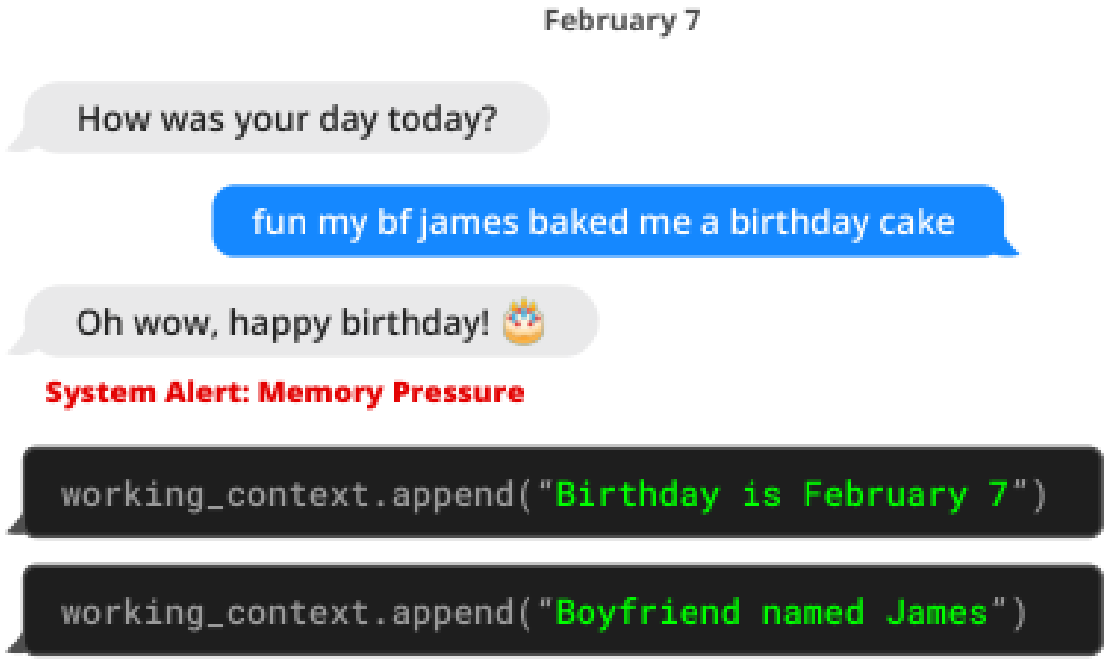
Persistent agent memory is a system capability.

Human memory gives us useful vocabulary

Human analogy	Agent mechanism	Example
Working memory	Active context	Current task and recent errors
Episodic memory	Interaction records	Last week's debugging session
Semantic memory	Saved facts and preferences	"Use pytest in this project"
Procedural memory	Learned behavior / instructions	A testing workflow

- An analogy for organizing systems, not a claim about human cognition
- Model weights and editable notes have very different update mechanisms

MemGPT: virtual memory for a language model



- Treat the context window like a small working area
- Keep details outside it; move selected information in when needed
- On a *memory pressure* alert, the agent writes facts out with function calls

Large backing storage does not make active context infinite.

Figure 1 of Packer et al., MemGPT (2023): https://arxiv.org/abs/2310.08560

Memory in a product is a lifecycle

```
Observe → decide what to save → store → retrieve → update or forget
```

- Chat assistants and coding agents can provide persistent memory
- Features and user controls differ by product
- Repeated corrections can become a preference—but should they?
- Saved facts need provenance, scope, and a way to correct them

Discussion: Is “use Python 3.11” a permanent preference or a project fact?

A memory file should say what it knows

Illustrative note schema:

```
---
name: test-workflow
type: project
source: repository README, checked 2026-10-14
---
Run unit tests with: python -m pytest tests/
Recheck the repository instructions if this command fails.
```

- Useful categories: user preference, feedback, project fact, reference
- Store a pointer when the authoritative detail lives elsewhere

Memory suggests where to look; current evidence confirms what is true.

Q3 takeaway: retrieval must reach the workbench



- Information in storage is not automatically available to the model
- Retrieval can miss, select the wrong fact, or return stale detail
- The harness decides what to include within the available budget

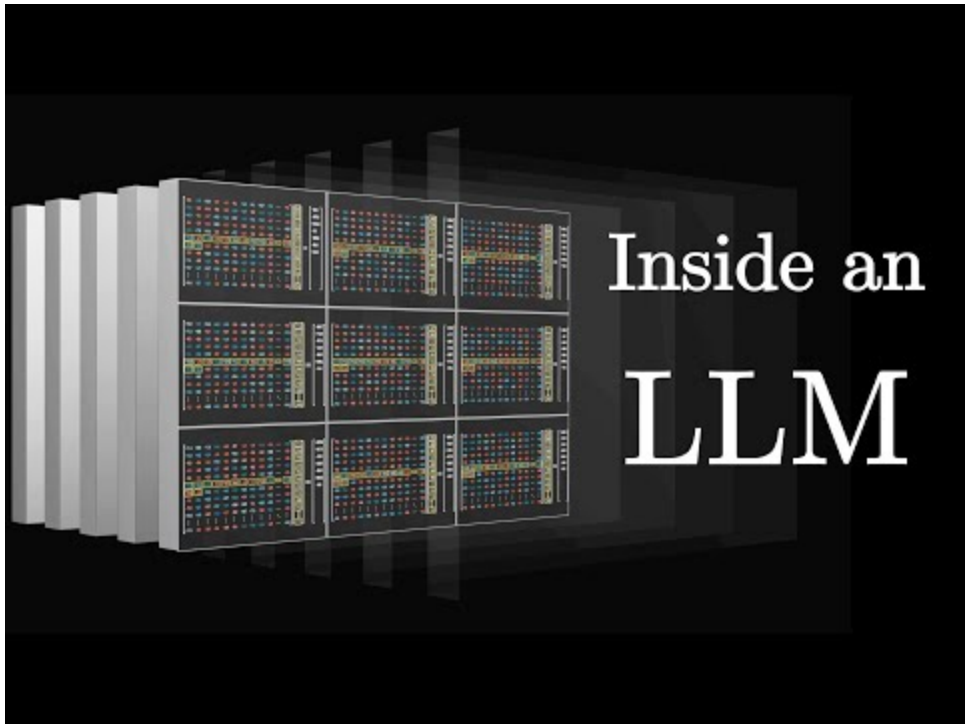
Distinguish “stored somewhere” from “usable right now.”

Q4: Why isn’t agent memory infinite?

Why not paste the entire codebase, every document, and every past conversation?

Two constraints: compute and answer quality

Video: how a transformer processes context



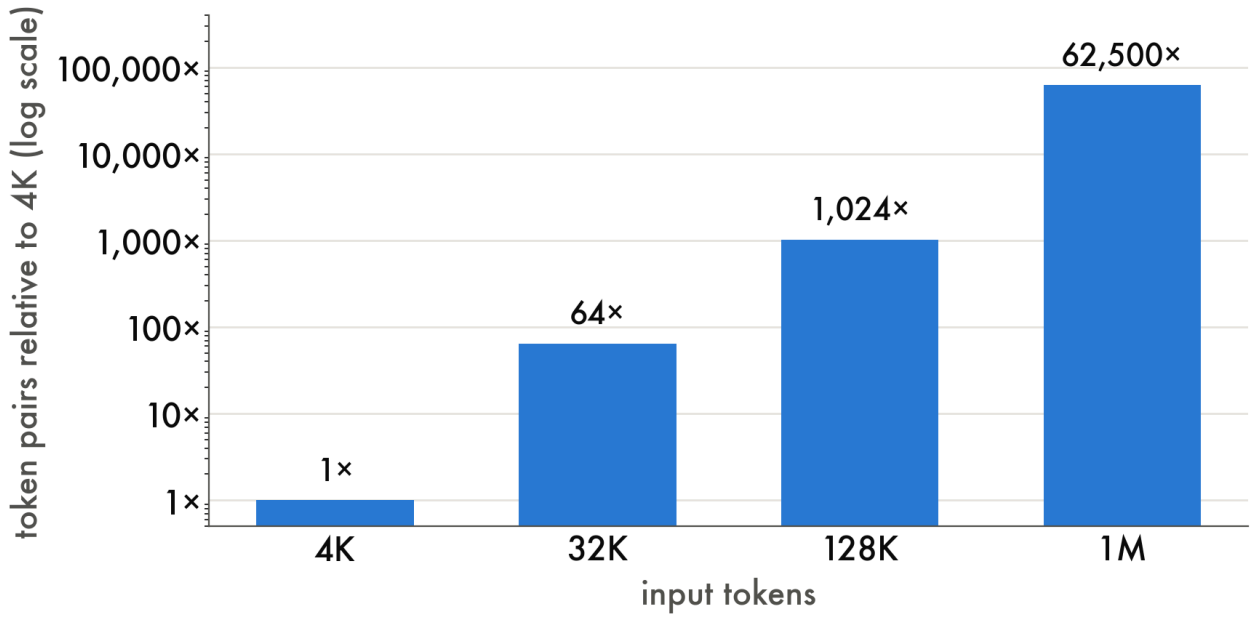
3Blue1Brown — Transformers, the tech behind LLMs

Watch: Deep Learning, Chapter 5

As you watch: what work grows when we add more input tokens?

3Blue1Brown. Transformers, the tech behind LLMs | Deep Learning Chapter 5. https://www.youtube.com/watch?v=wjZofJX0v4M

Constraint 1: dense attention grows quadratically

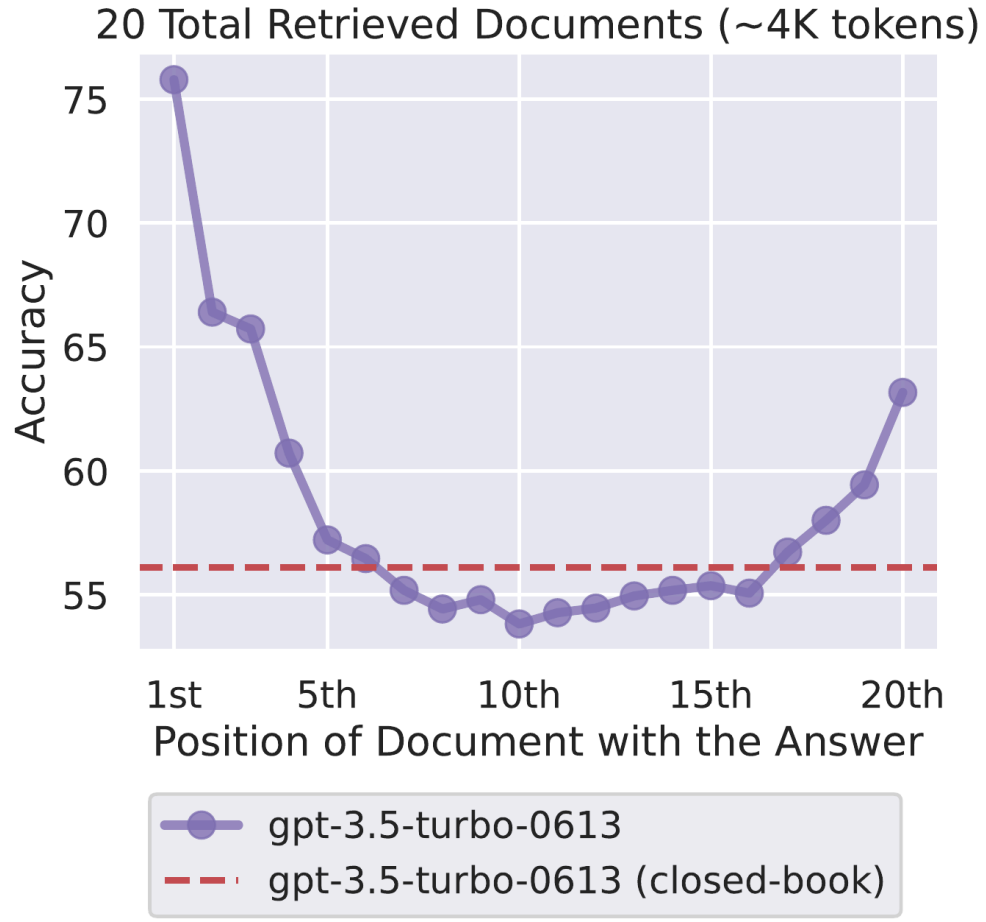


For standard full attention, the number of token-pair interactions is **$O(n^2)$**.

- Doubling the input quadruples the pairs
- **These are attention-pair ratios, not predictions of total runtime**
- Other operations, kernels, caching, and hardware also matter

Vaswani et al., Attention Is All You Need: https://arxiv.org/abs/1706.03762

Constraint 2: “fits” does not mean “gets used well”



Lost in the Middle: answer quality depends on where the relevant evidence sits.

- High at the start and the end, lower in the middle: a U-shape
- Position effects depend on model, task, and context length
- A larger advertised window does not guarantee reliable retrieval everywhere

Test with your own tasks; do not treat one curve as a universal law.

Figure 1 of Liu et al., Lost in the Middle (TACL 2024): https://arxiv.org/abs/2307.03172

Why might position affect retrieval?

Possible mechanisms to investigate:

- **Training distribution:** which positions contain useful evidence?
- **Position representations:** does extrapolation change behavior?
- **Distractors:** can the model select evidence among similar passages?
- **Attention allocation:** which tokens receive disproportionate attention?
- **Causal order:** what information was available at each encoded position?

These are hypotheses and model-dependent mechanisms, not six proven causes.

Similarity to human serial-position effects is an analogy, not an explanation.

Prompt order: what do our experiments support?

```
Instruction first: [task] [document]
Instruction last:  [document] [task]
```

- **Our experiments do not support the claim that instruction-first is better**
- Causal attention alone does not establish which ordering will perform better
- This result applies to the tested models, prompts, and counting task
- Compare accuracy, latency, and cache use separately

Treat prompt order as an empirical question, not a universal rule.

Optional: Advertised context vs. effective context

Capacity question	Quality question
Does the API accept this many tokens?	Can the model reliably use them?
What is the input/output limit?	What happens with distractors?
How much does the request cost?	Does multi-step retrieval still work?

- RULER evaluates more than finding one obvious needle
- Effective performance varies by task and model
- There is no universal “50–65% of the window” rule

Benchmark a useful operating range rather than filling the limit by default.

Hsieh et al., RULER (2024): https://arxiv.org/abs/2404.06654

Optional: Research changes different parts of the equation

Approach	What it targets
RoPE scaling / YaRN; ALiBi	Position handling and length generalization
FlashAttention	Attention memory traffic and kernel efficiency
Ring Attention	Distributing long-context attention
State-space models, e.g. Mamba	An alternative sequence-processing architecture
Mixture of experts	Conditional compute / parameter scaling

These are different mechanisms. MoE alone does not remove attention's context cost.

Optional: More context or less context?

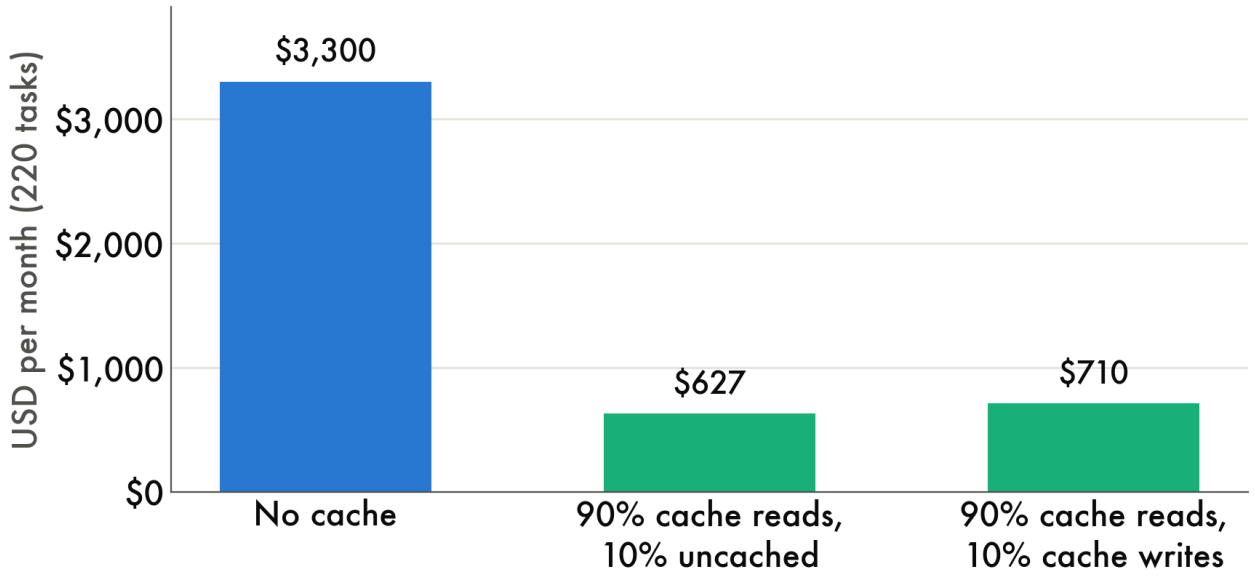
Include too little	Include too much
Missing requirements	Distracting or contradictory detail
Repeated searches	Repeated input cost
Wrong assumptions	Longer prefill and larger KV cache

Ask before adding a document:

1. What decision could it change?
2. Could a relevant excerpt or pointer suffice?
3. Can we retrieve it later if needed?

Optimize for relevant evidence, not token count alone.

Exercise: what does repeated context cost?



Assume **100 calls/task**, **50K input tokens/call**, **$3/M input**, 220 tasks/month; ignore output and tools.

- No cache: 100 × 50,000 × $3/M = **$15/task**
- 90% cache reads at 0.1×: **$2.85/task**
- If the remaining 10% are cache **writes** at 1.25×: $3.23/task

Actual hit rates and output tokens change the total substantially.

Claude API prompt caching: https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Quiz: same report, a new question

```
Request 1: [Report: 10,000 tokens] [Question 1: 20 tokens]
           “What were the company’s main expenses?”
Request 2: [Same report: 10,000 tokens] [Question 2: 20 tokens]
           “How did revenue change?”
```

Assume the report is an eligible, valid cached prefix. Each request contains only the shown blocks; Request 1’s answer is not included.

For Request 2, how many input tokens occupy context, and how many reuse cached computation?

Choice	Input tokens in context	Cached tokens reused
A	20	10,000
B	10,020	10,000
C	10,020	10,020
D	10,020	0

Quiz: what if we move the question first?

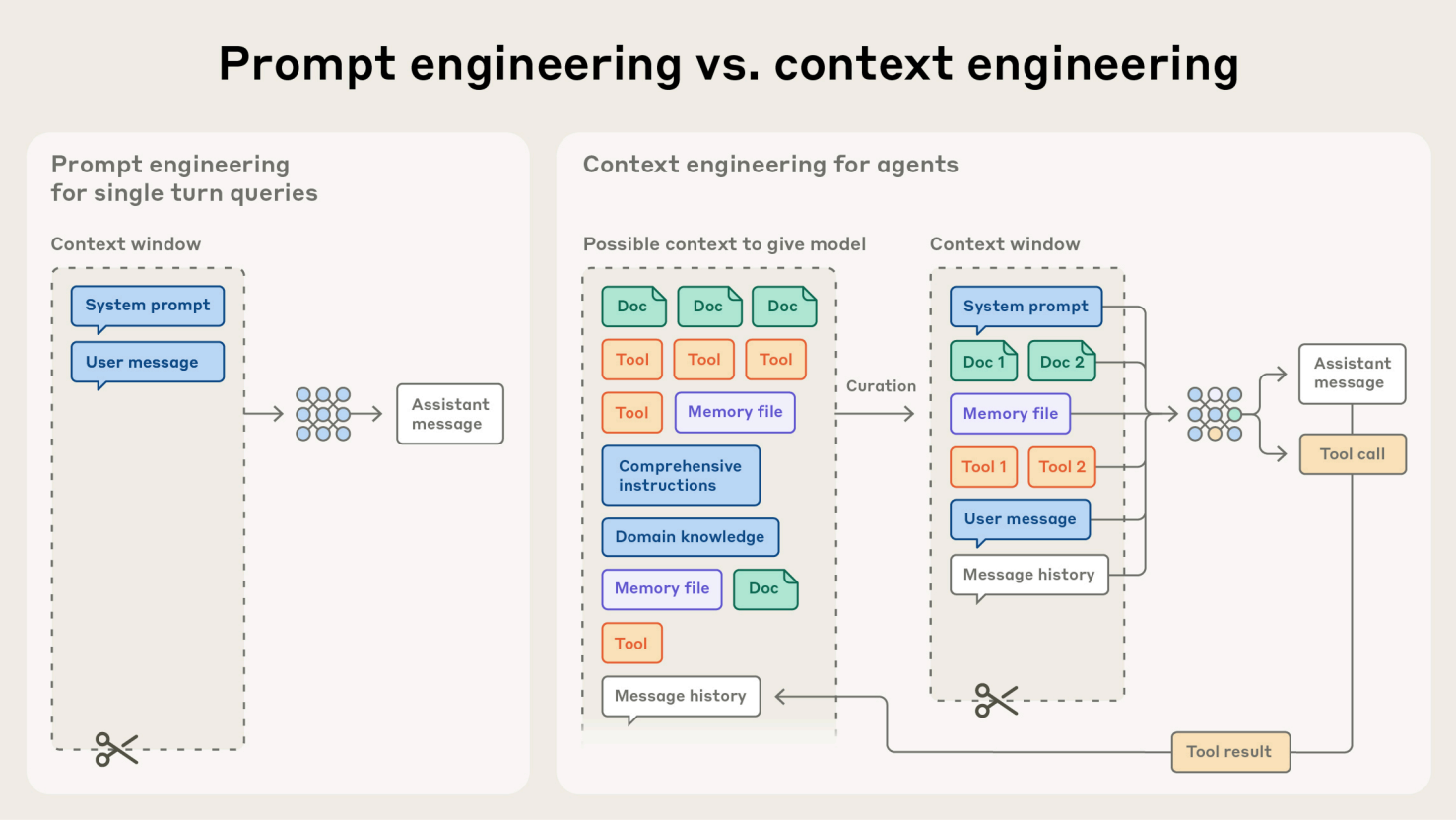
```
Request 1: [Report: 10,000 tokens] [Question 1: 20 tokens]
           “What were the company’s main expenses?”
Request 2: [Question 2: 20 tokens] [Same report: 10,000 tokens]
           “How did revenue change?”
```

Assume exact-prefix caching and no other matching cache entry. Each request contains only the shown blocks.

For Request 2, how many input tokens occupy context, and how many reuse cached computation?

Choice	Input tokens in context	Cached tokens reused
A	20	10,000
B	10,020	10,000
C	10,020	10,020
D	10,020	0

Q5 · From prompt writing to context engineering



Prompt writing asks how to phrase one instruction; context engineering asks what the model should see at this step.

Figure: Anthropic, Effective context engineering for AI agents (2025): https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Prompting before instruction tuning

A **base model** only continues text. Ask it a question:

```
What is the capital of France?
```

and it may continue the pattern:

```
What is the capital of Germany?
What is the capital of Italy? …
```

So prompts were written as **documents to complete**, with examples (**few-shot**):

```
English: cheese → French: fromage
English: house  → French: maison
English: book   → French:
```

The model completes `livre`.

Instruction tuning (InstructGPT, 2022) trained models on instruction–response pairs, so a plain request now just works.

Chain of thought: show the reasoning

```
Q: A juggler has 16 balls. Half are
   golf balls, and half of the golf
   balls are blue. How many blue
   golf balls are there?
A: Let's think step by step.
   16 / 2 = 8 golf balls.
   8 / 2 = 4 blue golf balls.
   The answer is 4.
```

- Ask for (or show) the **steps**, not just the answer
- *"Let's think step by step"* raised MultiArith accuracy from **17.7% to 78.7%**
- The intermediate tokens act as **working memory** in the context
- Today's models **think** before answering by default: chain of thought is built in

Wei et al., Chain-of-Thought Prompting, NeurIPS 2022 · Kojima et al., Large Language Models are Zero-Shot Reasoners, NeurIPS 2022

What the model actually reads

```
[system]    You are a coding agent. Ask before destructive commands.   ← system
            Tools: read_file(path), edit_file(…), bash(cmd)            ← system
            # CLAUDE.md: run tests with python -m pytest               ← system
            # Memory: user prefers small, reviewable diffs            ← system
            Date: 2026-10-19 · cwd: ~/projects/parser                 ← system
[user]      Find out why the tests fail, fix the bug, verify the fix.  ← you
[assistant] (the model continues from here)
```

- You typed ~15 tokens; the model reads **thousands**, mostly written by the **system** (the harness)
- **System prompt:** role, rules, tools, project files, memory, date; **user prompt:** your instruction, plus tool results fed back later
- Roles are formatting the model was **trained** to respect: it is all one token sequence

Six ingredients of a model request

Component	Example
System instructions	Role, constraints, response format
Tool definitions	Names, schemas, usage guidance
Examples	A representative input/output pair
Message history	Recent decisions and user corrections
Retrieved data	Relevant source files or documentation
Agent memory	Durable project facts and preferences

Each component competes for the same finite input budget.

Prompt engineering vs. context engineering

	Prompt engineering	Context engineering
Main object	An instruction or template	The assembled model request
Timing	Often designed before the task	Updated throughout the task
Decisions	Wording, examples, format	Retrieval, tool results, history, budgets
Failure	Ambiguous request	Missing, stale, or excessive evidence
Analogy	Writing an email	Maintaining the working file for a case

They complement each other: a well-selected context still needs clear instructions.

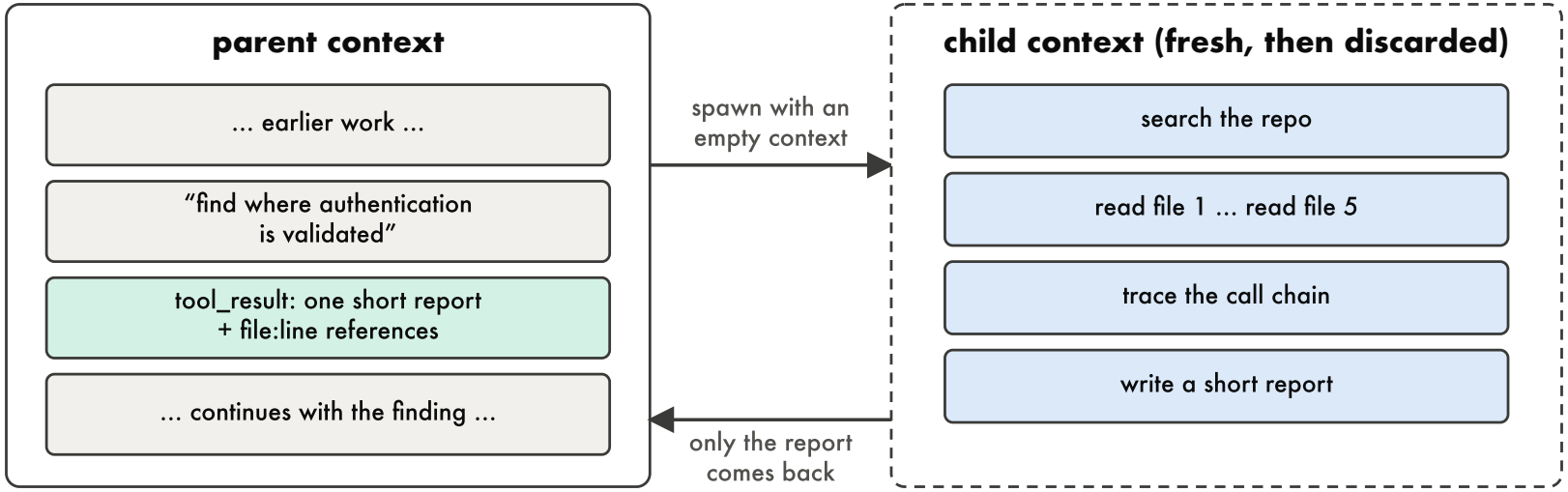
Five ways to stretch a finite context

1. **Compaction:** summarize earlier interaction
2. **Structured notes:** save durable findings outside the conversation
3. **Subagents:** investigate in separate contexts; return concise findings
4. **Efficient tools:** return focused results rather than enormous dumps
5. **Just-in-time retrieval:** load details when the next step requires them

Move detail out of active context, while keeping a way to recover it.

Anthropic, Effective context engineering for AI agents (2025): https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Subagents create a context boundary



- Reports should carry evidence and uncertainty; delegation adds model calls, latency, and coordination overhead

A short report can save parent context while losing important detail.

A real agent can have a small core

mini-SWE-agent demonstrates a compact model/action/observation loop.

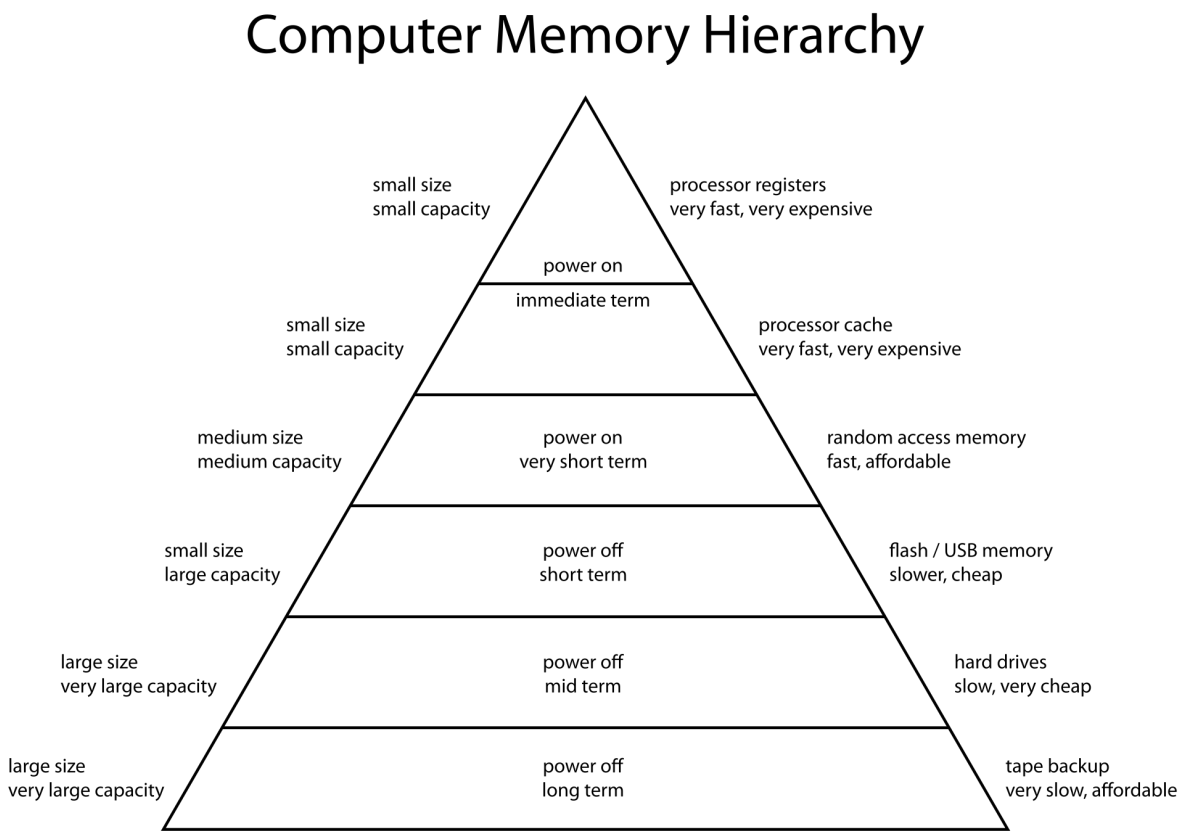
```
messages → model → shell action → observation → messages → ...
```

- The project describes a roughly 100-line core and reports >74% on SWE-bench Verified
- That result depends on the evaluated model and setup
- A simple architecture can still retain a growing linear history
- Keeping only the last 10 messages would be a separate teaching policy

Small orchestration code does not imply small total system complexity.

Project README and implementation: https://github.com/SWE-agent/mini-swe-agent

Recall L1: a small fast level, a large slow level

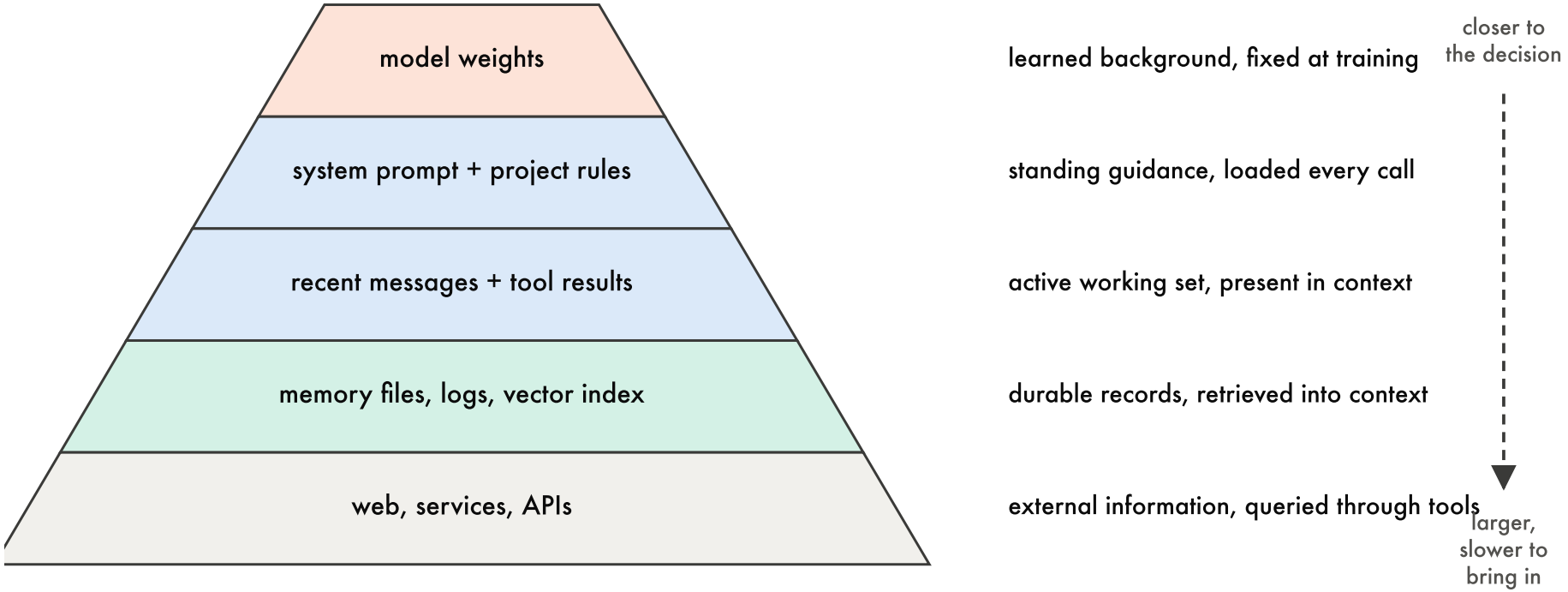


- Registers and caches: small, fastest, volatile
- DRAM: larger, slower, volatile
- SSD and disk: much larger, slower again, persistent
- Remote storage: enormous, network-bound

Keep actively used data close to computation; move colder detail to larger backing storage.

What is the agent's equivalent of an active working set?

An agent's information hierarchy



A functional analogy, not literal L1/L2 hardware. Weights are not registers; text in context has no tier-specific latency.

How project instructions reach the model

```
Managed guidance → user guidance → project guidance → local guidance
                                                    + applicable scoped rules
```

- Ancestor instruction files can load at session start
- Subdirectory instructions and scoped rules can load when relevant
- Contents guide the model; they are not enforced access controls
- Conflicting text is unreliable: resolve contradictions explicitly

Do not assume “the last instruction always wins.”

Claude Code memory documentation: https://code.claude.com/docs/en/memory

What belongs in CLAUDE.md?

```
# Development
- Install: python -m pip install -e '.[dev]'
- Test: python -m pytest tests/

# Project map
- src/parser/: parsing implementation
- tests/fixtures/: sample input files

# Conventions
- Keep parsing independent of network access.
- Add a regression case when fixing a parser bug.
```

Prefer concise, verifiable project guidance over a copy of the codebase.

Different products, similar instruction files

Mechanism	Purpose
`AGENTS.md`	Repository guidance for compatible coding agents
`CLAUDE.md`	Claude Code project and user guidance
Copilot repository instructions	Repository-specific coding guidance
Cursor rules	Scoped instructions in the editor
Chat-product custom instructions	Standing user preferences

Common content: commands, structure, style, testing, constraints.

Formats, discovery rules, and enforcement differ across tools.

Translate the cache vocabulary carefully

Hardware term	Agent analogue
Hit / miss	Useful note found / search needed
Cold start	No saved observations for a project
Capacity pressure	Active context approaching its budget
Write-back	Persist an observation to a durable record
Prefetch	Load likely-needed guidance in advance
Eviction	Remove details from the active context

A prompt-cache hit reuses computation; a memory hit retrieves information.

Four levers for reducing active context

Lever	Preserve	Move out or replace
Save large output to a file	Path and relevant excerpt	Full command output
Clear older tool payloads	Decisions and recoverable references	Bulky observations
Automatic compaction	Current goal, constraints, progress	Earlier conversation detail
Manual compaction	A deliberate task handoff	Unneeded interaction history

Summaries are lossy: save critical details before discarding them.

A pointer helps only if the original evidence remains accessible.

Context tooling: https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools

Arrange stable context for prompt-cache reuse

```
[stable rules | stable tools | stable reference] [changing turn]
                cacheable prefix                        suffix
```

- An eligible matching prefix avoids repeating its prefill work
- A changed prefix or expired entry can trigger new computation
- Anthropic supports 5-minute and 1-hour cache lifetimes
- Savings depend on model rates, writes, reads, and uncached suffixes

Cache reuse does not increase the model's context limit.

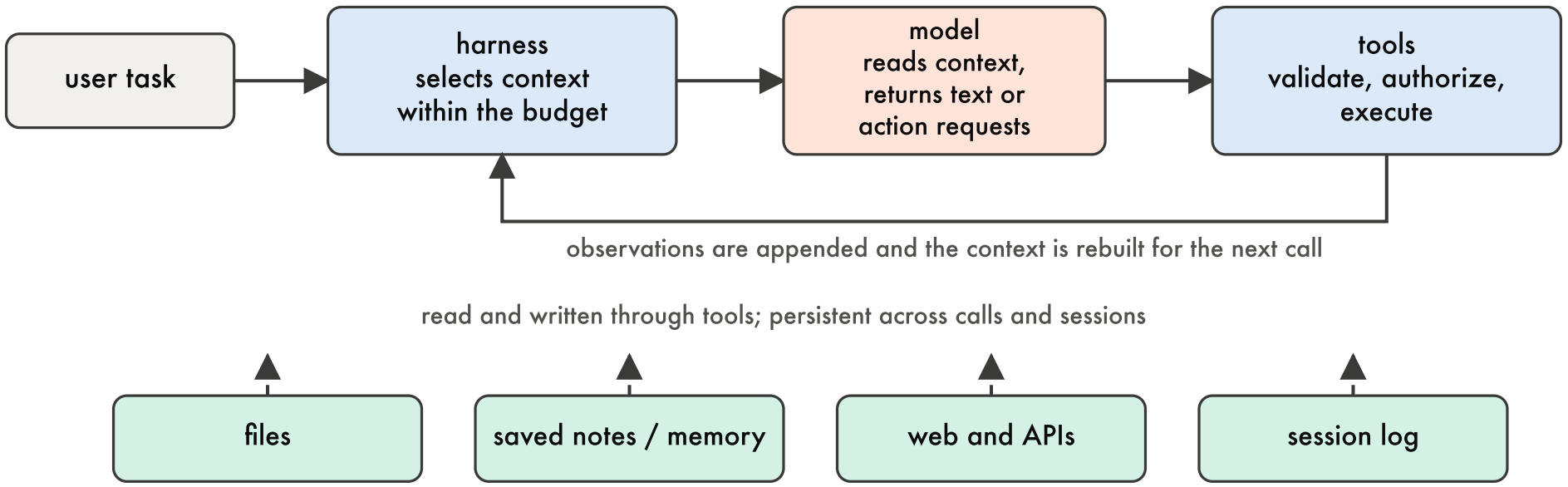
Claude API prompt caching: https://platform.claude.com/docs/en/build-with-claude/prompt-caching

When the memory hierarchy fails

Failure	Example	Repair
Oversized standing context	Every task loads the whole handbook	Scope or retrieve guidance
Contradictory rules	Two incompatible test commands	Reconcile instructions
Stale memory	Old API behavior saved as current fact	Recheck authoritative evidence
Context thrashing	Same files repeatedly reread	Keep a concise working summary
Cold start	No project map or test command	Build a small verified index

Saving more is useful only when selection and updates work.

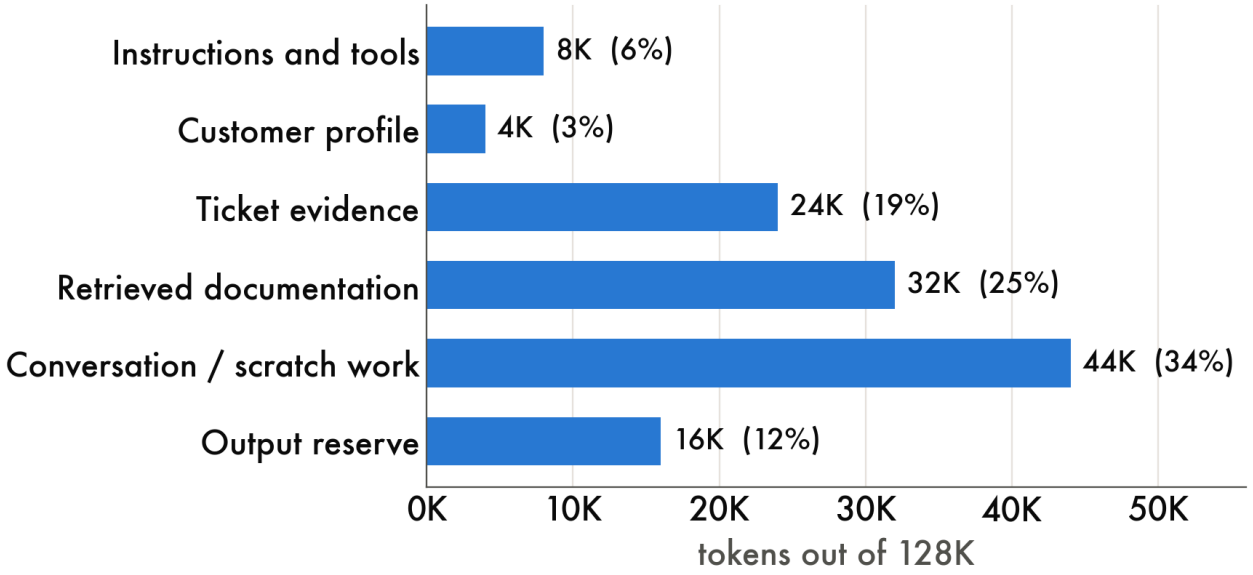
The full information flow



the context window is the bottleneck: design the movement of information, not just its storage

- Context selection limits what the model can use on this call; tool bandwidth and retrieval quality limit what arrives next
- Persistent records let the system resume without loading everything

Exercise: allocate a 128K-token workbench



A hypothetical budget for a **support agent**, including space reserved for output.

- Standing context is small; evidence and history dominate
- Output reserve is part of the budget, not an afterthought

Pairs: rebalance for a coding agent. What stays loaded, and what moves to retrieval?

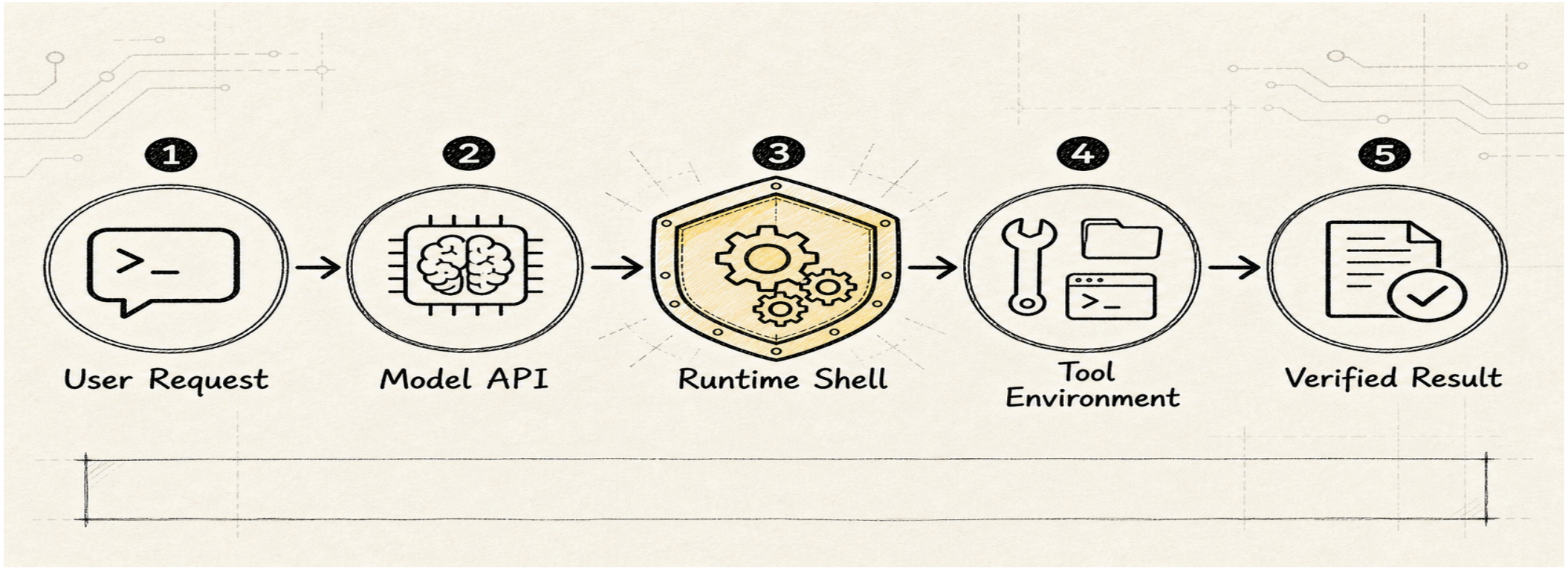
Q6: Inside the Claude Code harness

A deep dive through the source-code leak

- March 2026 reporting described source exposure through an npm source map
- Follow Chapters 1–5: architecture, loop, prompts, context, and tools
- Follow one task: **“Find the failing test, fix it, and verify.”**

Leak reporting: https://www.theregister.com/software/2026/03/31/anthropic-accidentally-exposes-claude-code-source-code/5227940 · Source analysis: LienJack, DEV Community, Chapters 1–5.

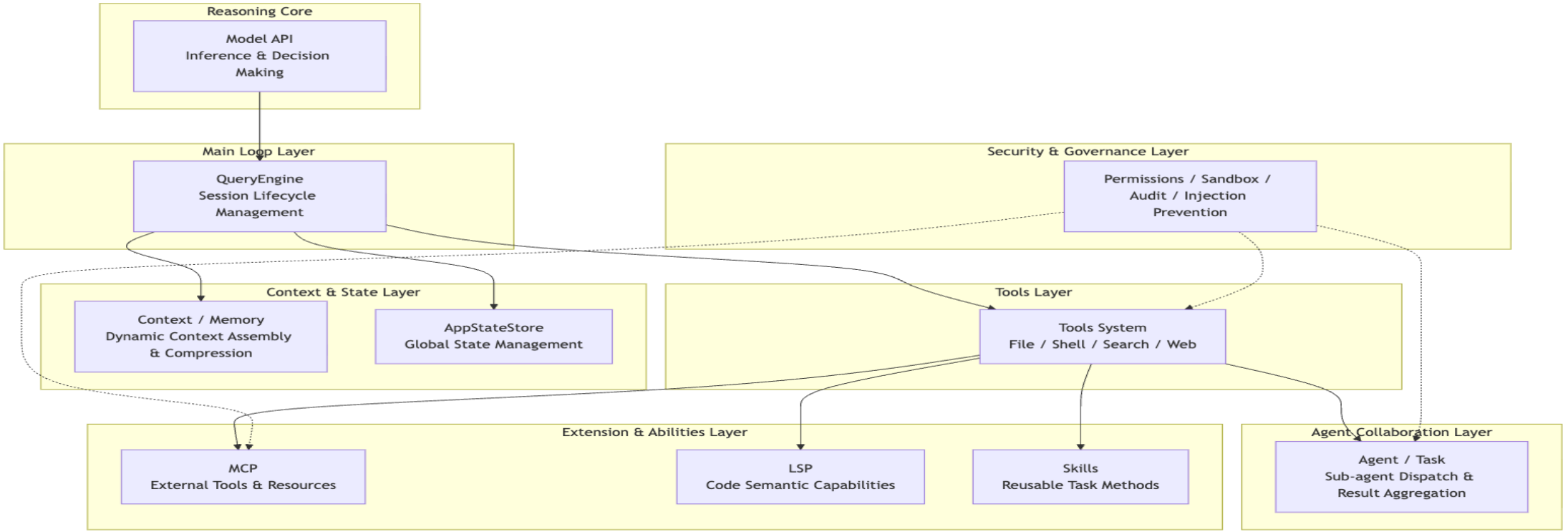
What the exposed application code helps explain



The model proposes actions; the host executes them and records results.

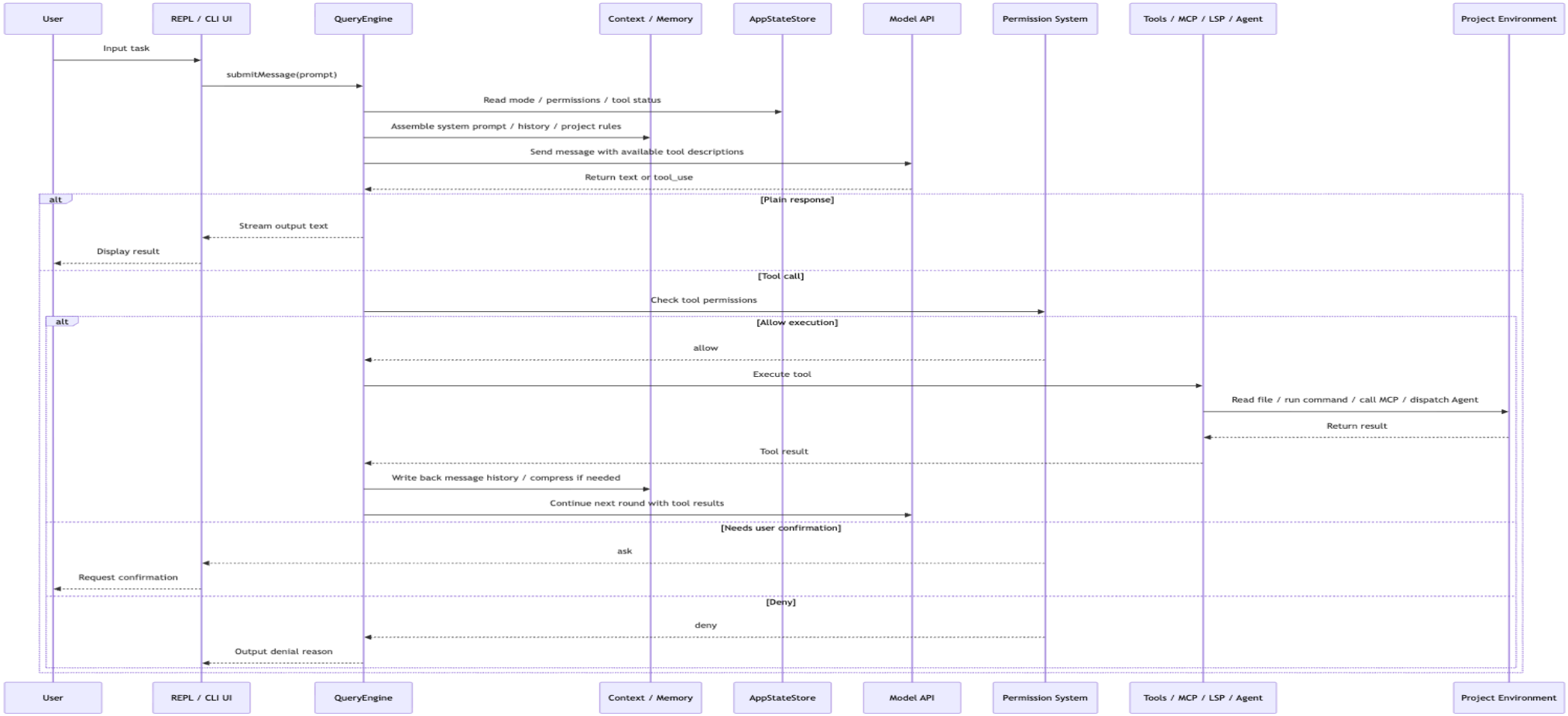
LienJack, Claude Code Source Analysis, Ch. 1: Architecture. https://dev.to/lien_jp_db54b8b7fd9fa0118/claude-code-source-analysis-series-chapter-1-architecture-48d7

Read the harness as connected subsystems



Orchestration connects tools, context, state, extensions, and permissions.

Trace submitMessage() through the runtime



LienJack, Claude Code Source Analysis, Ch. 1: Architecture. https://dev.to/lien_jp_db54b8b7fd9fa0118/claude-code-source-analysis-series-chapter-1-architecture-48d7

QueryEngine.ts vs. query.ts: session and loop

Reported component	Responsibility	Failing-test example
`QueryEngine.ts`	Carries session history, tools, caches, and permission context	Retain which files were read and which actions were denied
`query.ts`	Drives model calls and tool-result feedback	Request tests → observe failure → edit → rerun

- **Action loop:** model request → tool execution → observation
- **State loop:** changed history and session state → next context

A single user message can trigger several model/tool cycles.

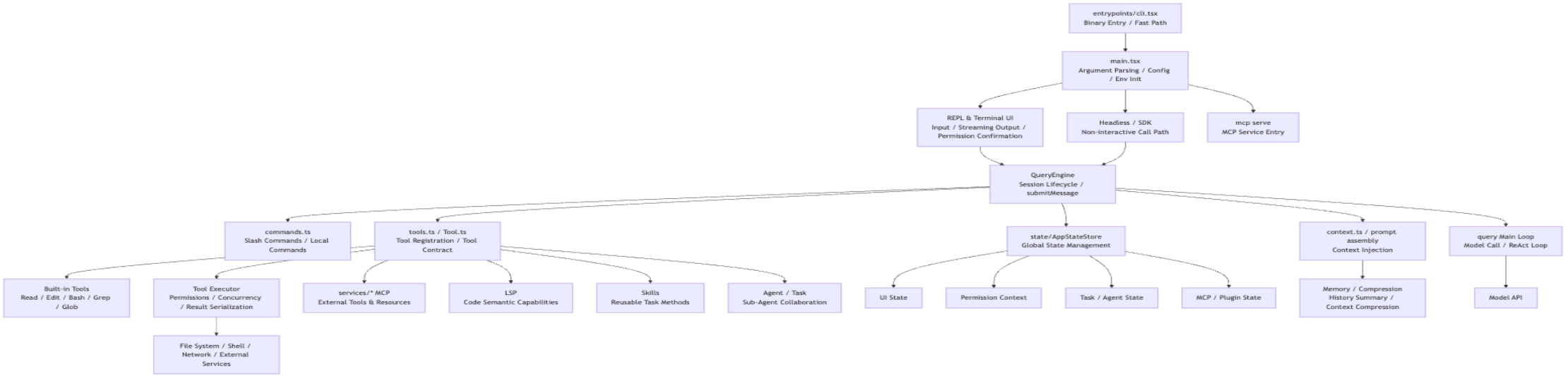
Tool.ts: a request must become a valid action

```
Model tool_use
  → schema + input validation
  → permission checks + hooks
  → execution
  → serialized tool_result → next model call
```

- `Tool.ts` : describes a tool's contract and execution properties
- `tools.ts` : determines the available tool menu
- `services/tools/toolExecution.ts` : coordinates execution

Reading a file and running a shell command need different host-side controls.

Follow the source map: entry → loop → context

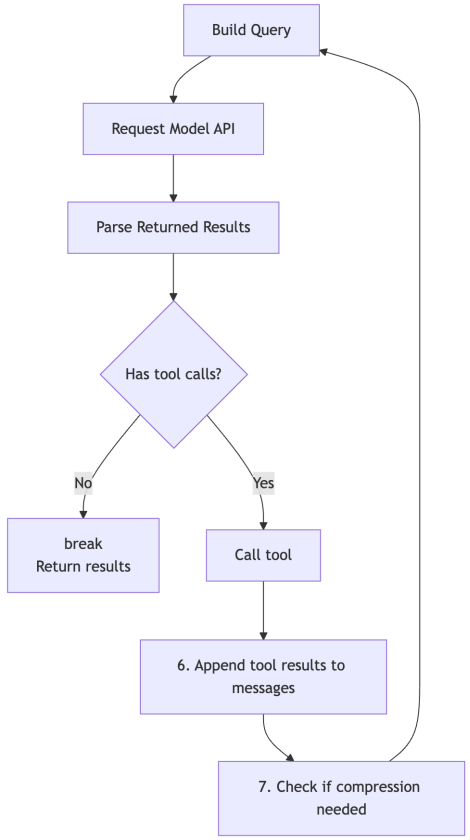


`cli.tsx` / `main.tsx` → `QueryEngine` / `query` → tools

`context.ts` + prompts + compaction determine what reaches the model.

Names in this diagram describe the author's analysis, not a stable public API.

Chapter 2 · ReAct is a state machine



- Tool requests take the execution branch
- Results re-enter the message history
- Context pressure is checked before continuing

Trace what survives into the next iteration.

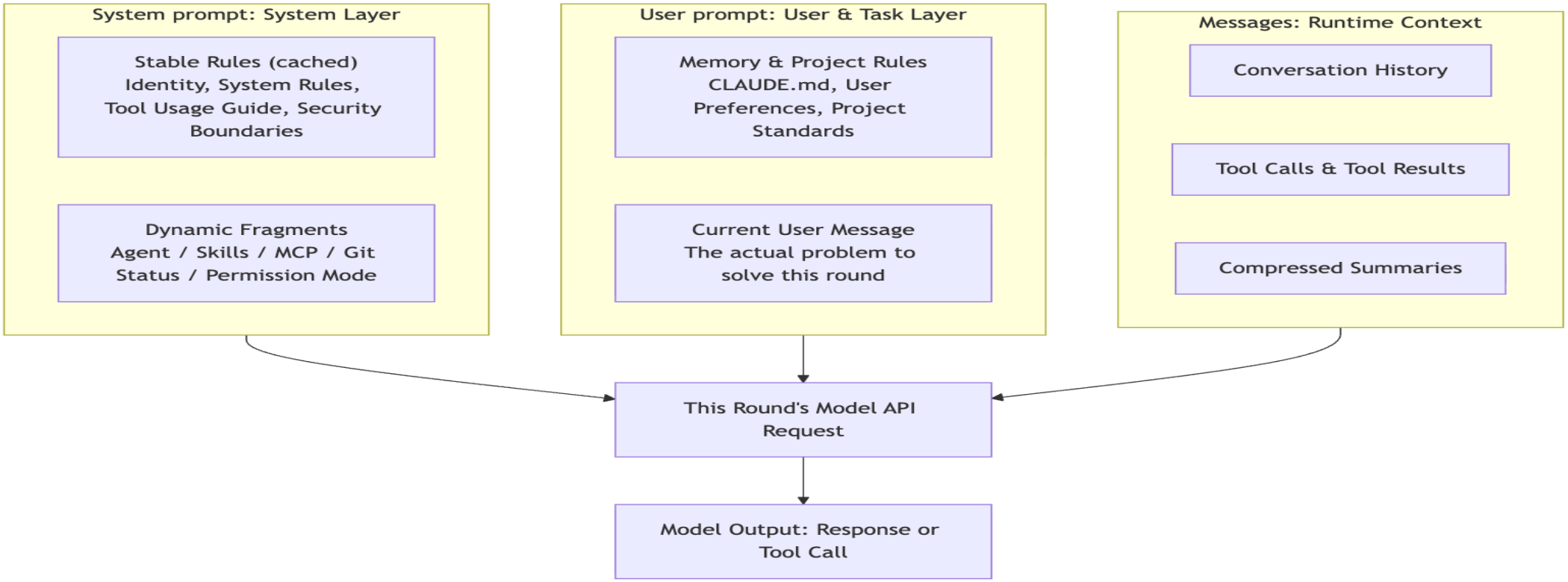
Chapter 2 · What must survive a loop iteration?

Reported state	Why it matters
`messages`	Carries requests, decisions, and observations
`toolUseContext`	Supplies the execution environment
`turnCount`	Tracks iteration progress
Compaction tracking	Records context pressure and failures
Abort state	Allows interrupted work to stop

Example: after a failed test, carry the error forward before choosing a fix.

A useful loop needs limits, cancellation, and a way to recover—not just repetition.

Chapter 3 · Build the request, not just a prompt



Separate standing guidance from changing observations.

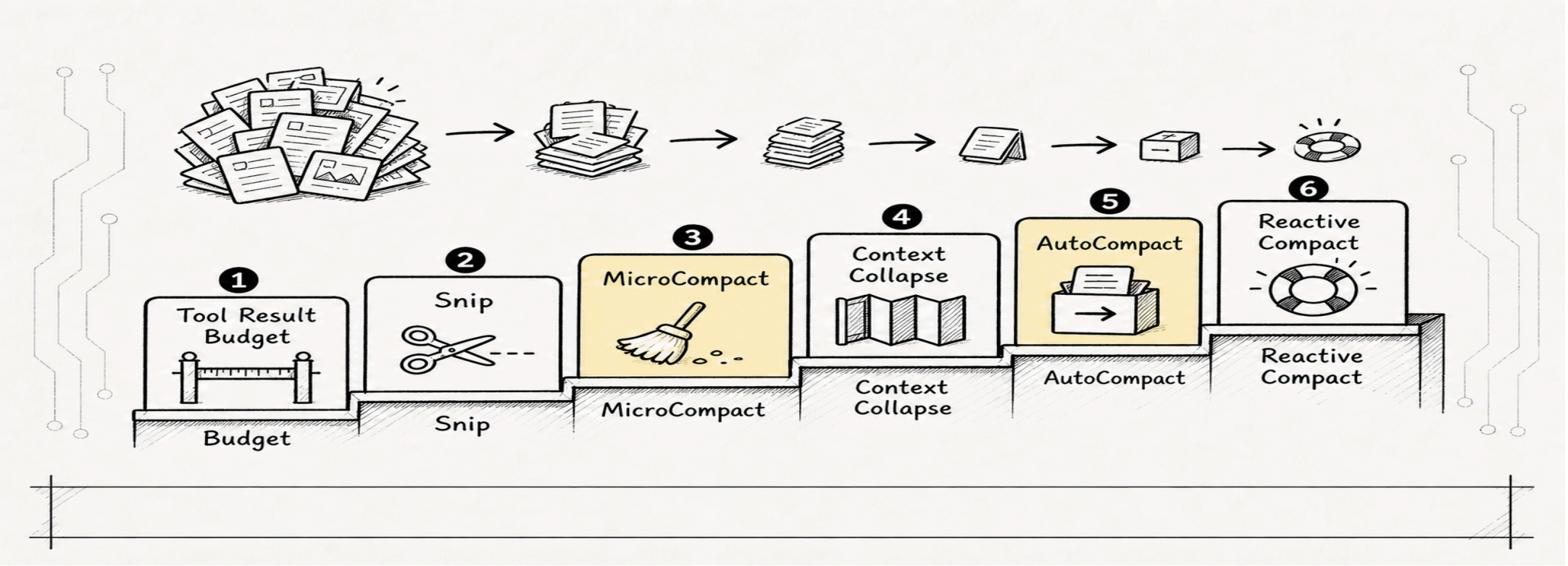
Chapter 3 · Selection, assembly, and cache boundaries

```
Select main system prompt → append supplementary guidance
                  + applicable project / user memory
                  + current environment and tools
                  + messages and tool observations
                  → this turn’s request
```

- Replacement and appended prompts have different effects
- Keep stable content separate from changing runtime information
- Tool results become input to the next request

Example: project test guidance stays relevant; the newest failure log changes the next step.

Chapter 4 · Escalate context cleanup gradually



Use lighter cleanup before replacing history with a summary.

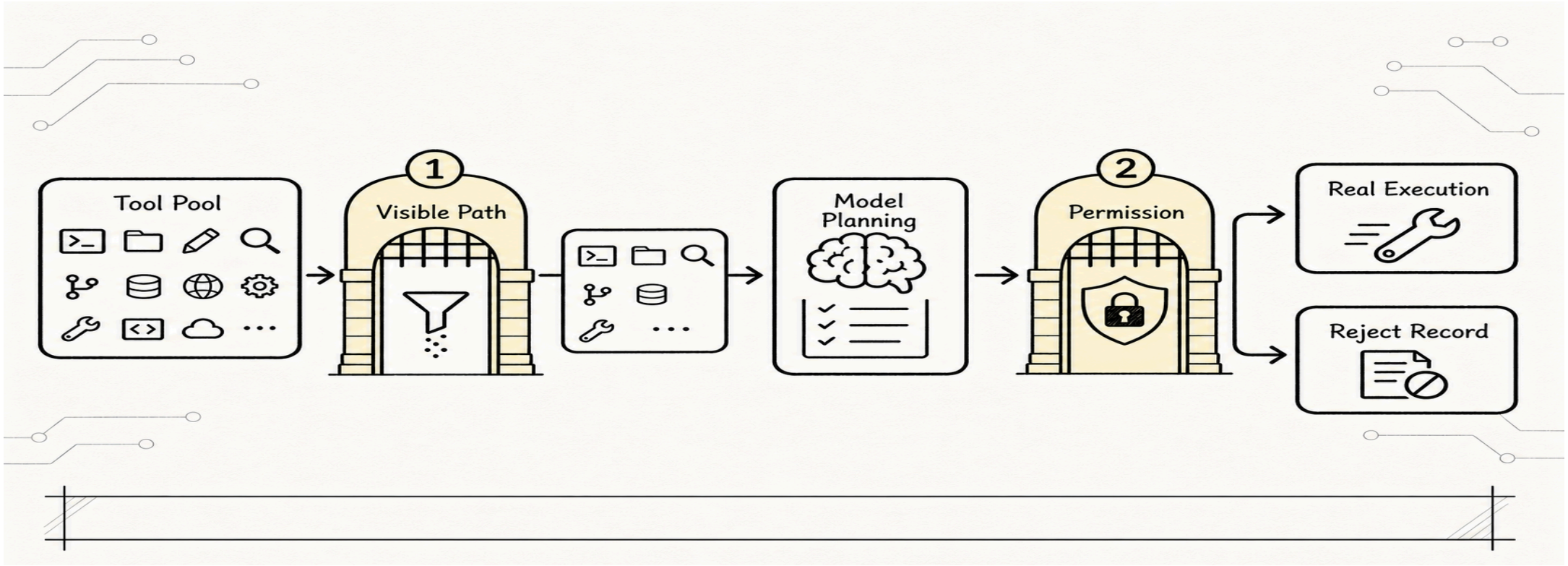
Chapter 4 · Preserve the task while shrinking history

```
Budget tool outputs → snip bulk → micro-compact stale results
     → collapse the view → auto-compact when needed
                     → reactive compaction after overflow
```

- Keep tool-call/result structure intact during cleanup
- A handoff summary needs the goal, constraints, attempts, and next step
- Retain recent turns and observations alongside the summary

Example: replace an obsolete file read; preserve the latest error and “do not change the public API.”

Chapter 5 · Tools pass through two gates



Visibility and permission to execute are separate decisions.

Chapter 5 · A registry is not the model's tool menu

```
Candidate registry → mode / policy filtering → visible tools
                                                  ↓ model request
Validation → hooks + permission checks → scheduling → execution
                                                  ↓
                                        result → message history
```

- `getAllBaseTools()` supplies candidates, not final availability
- A tool's contract includes result limits and concurrency properties
- Independent reads may run together; dependent edits need ordering

Example: seeing `Bash` does not authorize every possible shell command.

Source-reading check: where does execution happen?

The model returns a request to run `python -m pytest`.

Which component executes the command and supplies its output to the next model call?

A. The model's weights
B. The tokenizer
C. The harness's tool-execution path
D. The prompt cache

Q6 takeaway: the harness is a software system

```
Agent = model + context management + tools + state + execution control
```

- Trace **entry → session → loop → tool → observation**
- Trace **context assembly → compaction → persistent state**
- Use the analyzed snapshot to understand responsibilities and interfaces

A strong model and a reliable harness solve different parts of the task.

What should you take away?

1. A URL request crosses layers that can reuse prior work
2. An agent request also crosses a system, not just a model
3. Persistent memory needs storage, retrieval, and updates
4. The remaining constraints are compute and answer quality
5. A hierarchy keeps useful evidence near the current decision
6. The source-analysis case study exposes the harness's session, loop, tool, and context responsibilities

Revision to the Syllabus:

1. Memory Hieraching.
2. Process Management.
3. Parallelism (Task, Data)
4. Data flow systems
5. Model training.